

数据挖掘与机器学习

第一次作业

信管 2301

阮钰博

2021321054017

内容概览

数据挖掘与机器学习核心知识体系梳理，从数据准备到模型评价的完整技术路径。

01 准备数据

数据格式规范、变量类型识别、特征工程与缺失数据处理，奠定高质量分析基础

02 选择算法

无监督学习、监督学习与强化学习三大范式，匹配不同业务场景与任务目标

03 参数调优

理解过拟合与欠拟合的成因，通过正则化技术优化模型复杂度与泛化能力

04 评价模型

分类与回归指标体系、混淆矩阵解析、交叉验证方法，客观衡量模型真实价值

05 概念汇总

大数据 4V 特征、学习范式对比、数据预处理技术体系，构建完整知识图谱

数据准备：基础规范与变量类型

数据质量决定分析上限。表格是最常用的数据存储格式，其中行代表观测样本，列代表特征变量。准确识别变量类型是后续特征工程和算法选择的前提。

表格结构基础

行（Row）代表单个数据点或观测结果，列（Column）代表变量、特征或属性，这种结构化存储是机器学习算法的标准输入形式。

二值变量

仅有两种互斥取值（如是 / 否、买 / 不买），通常编码为 0 和 1，是分类任务中最简单的变量形态。

分类变量

包含多个离散类别（如顾客类别、颜色、地区），需通过独热编码等技术转化为数值形式才能被算法处理。

整型与连续变量

整型变量取整数值（如购买数量、点击次数），连续变量则取精细小数（如支出金额、温度、血压值），后者往往蕴含更丰富的信息量但对异常值更敏感。

数据准备：变量选择、特征工程与缺失值

核心洞察： 变量选择通过筛选相关特征避免维度灾难；特征工程通过创造性转换生成更具预测力的新特征，是提升模型性能的关键杠杆；缺失数据处理需在信息保留与计算效率间权衡，近似填充适用于随机缺失，预测填充适用于系统性缺失，删除仅作为最后手段。

01

变量选择

是一个试错过程，需借助相关性分析、特征重要性评估等方法，从原始特征中筛选出与目标变量相关性高、冗余度低的子集，以降低模型复杂度并提升泛化能力。

02

特征工程

通过对原始变量进行重新编码、组合或转换（如将多维度健康指标合并为主成分），生成更能揭示业务本质的新特征，往往比调参对模型性能的提升更显著。

03

近似填充法

分类 / 二值变量宜用众数填充，数值变量宜用中位数填充。计算高效但可能引入偏差，适用于随机缺失场景。

04

预测填充与删除

基于监督学习的预测填充法利用其他完整特征建立预测模型来估算缺失值，准确度更高但计算成本大；删除含缺失值的样本仅在缺失比例极低时适用，否则会导致样本偏差。

选择算法：三大学习范式

算法选择需匹配任务目标与数据特性。三种范式并非互斥，常在实际项目中组合使用。

无监督学习

在无标签数据中发现隐藏模式或内在结构，典型任务包括聚类（客户分群）、降维（可视化高维数据）和关联规则挖掘。

应用：根据购物记录对顾客自动分群；发现啤酒与尿布等关联商品组合。

监督学习

利用已知的输入 - 输出映射关系训练模型，必须依赖带标签的历史数据。分为回归与分类两大任务类型。

应用：回归任务预测房价、销售额；分类任务识别垃圾邮件、图像分类。

强化学习

通过与环境的持续交互和反馈（奖励 / 惩罚）来学习最优决策策略，部署后仍能根据新反馈自我迭代优化。

应用：AlphaGo 围棋程序、在线广告投放动态策略、自动驾驶决策优化。

参数调优：过拟合、欠拟合与正则化

参数调优的本质是在模型复杂度与泛化能力间寻找最优平衡。过拟合源于模型过度记忆训练数据噪声，表现为训练集表现极佳但测试集表现差；欠拟合源于模型容量不足，未能捕捉数据基本规律。正则化通过引入复杂度惩罚项约束模型参数，是防止过拟合、提升泛化能力的核心技术手段。

⚠ 过拟合 Overfitting

模型在训练集上准确率极高，但在未见过的新数据上预测效果显著下降，根源是模型过度学习了训练数据中的噪声和特化模式而非普遍规律。

■ 欠拟合 Underfitting

模型在训练集和测试集上表现均不佳，根源是模型过于简单或特征工程不足，未能充分捕捉数据中的内在结构和趋势。

✓ 正则化 Regularization

在损失函数中增加与模型复杂度成正比的惩罚项（如 L1/L2 范数），迫使模型在拟合数据与保持简洁之间权衡，有效降低过拟合风险。

⚙ 调参策略

需结合交叉验证评估不同参数组合下的泛化性能，避免在单一验证集上过度优化导致的“调参过拟合”。

评价模型：指标体系与验证方法

核心洞察：模型评价必须基于未参与训练的新数据，真实反映泛化能力。分类任务需警惕准确率在不平衡数据中的误导性，混淆矩阵能揭示模型在各类别上的具体表现差异。回归任务中 RMSE 对异常值敏感，适合惩罚大误差场景。交叉验证通过多次划分取平均，显著降低验证结果的方差，是小数据集下的标准验证策略。

01 分类任务的准确率陷阱

预测准确率 (Accuracy) 虽直观但可能掩盖严重问题，如在 95% 负样本的数据集上，始终预测负类也能达到 95% 准确率，因此必须结合精确率、召回率、F1 值等指标综合评估。

02 混淆矩阵的精确诊断

混淆矩阵 (Confusion Matrix) 将预测结果划分为 TP、FP、TN、FN，能精确定位模型在哪些类别上存在识别偏差，指导针对性优化。

03 回归任务的 RMSE 敏感度

均方根误差 (RMSE) 通过对预测误差取平方再开方，放大较大误差的影响，适合对异常值敏感、大误差代价高的业务场景 (如金融风控、医疗预测)。

04 K 折交叉验证的黄金标准

K 折交叉验证 (K-Fold CV) 将数据集分成 K 个子集，轮流使用 K-1 个子集训练、1 个子集验证，重复 K 次并取平均性能，显著降低数据划分随机性对评估结果的影响。

小结：数据科学研究的四个关键步骤

数据科学研究遵循严谨的四步流程：准备数据确保输入质量，选择算法匹配任务需求，参数调优平衡拟合与泛化，评价模型验证真实价值。四个步骤环环相扣，前一步的输出质量直接决定后一步的性能上限，构成从原始数据到业务价值的完整转化链路。

01 准备数据

完成数据清洗、格式转换、变量类型识别、特征工程与缺失值处理，高质量输入是后续所有分析的前提保障。

02 选择算法

明确任务类型（描述 / 预测 / 决策）与数据标签 availability，在无监督、监督、强化学习三大范式中选择最契合业务场景的算法族。

03 参数调优

通过交叉验证评估不同超参数组合，利用正则化技术控制模型复杂度，在经验风险与结构风险之间寻找最优权衡点。

04 评价模型

采用与业务目标对齐的评估指标（分类 / 回归），通过独立的测试集或交叉验证估计泛化误差，确保模型在实际部署中的可靠性。

概念汇总：大数据与数据挖掘

大数据的 4V 特征定义了现代数据环境的挑战与机遇。数据挖掘是从海量、嘈杂、不完全的数据中提取隐含模式与 actionable insights 的过程，其价值在于将原始数据转化为业务知识，支撑决策优化与模式发现。

Volume (体量巨大)

数据规模从 GB 级跃升至 PB 甚至 EB 级，传统单机存储与计算架构面临瓶颈，催生分布式存储 (HDFS) 与并行计算 (MapReduce/Spark) 技术栈。

Variety (类型繁多)

涵盖结构化数据 (关系型数据库)、半结构化数据 (XML/JSON 日志) 与非结构化数据 (文本、图像、音视频)，要求多模态数据融合处理能力。

Value (价值密度低)

有价值信息往往隐藏在大量冗余数据之中，需要通过聚类、分类、关联分析等挖掘技术萃取 " 数据石油 "，实现从数据到知识的跃迁。

Velocity (处理速度快)

业务场景要求流式实时处理 (如实时推荐、欺诈检测)，推动计算范式从批处理 (Batch) 向流处理 (Stream) 演进，数据时效性成为核心竞争力。

概念汇总：数据类型与学习范式

Key Insight: 数据类型决定预处理策略：结构化数据可直接输入算法，非结构化数据需经特征提取或深度学习编码。学习范式的选择取决于标签 availability 与业务目标——有监督学习追求预测精度，无监督学习探索未知模式，二者在实战中常结合使用。

数据类型

结构化数据

严格遵循预定义的数据模型（如关系型数据库表），每行每列都有明确类型定义，可直接用于传统机器学习算法

半结构化数据

具备一定层次或标记（如JSON、XML、日志文件），需通过解析转换为结构化形式，兼具灵活性与部分规范性

非结构化数据

无预定义数据模型（如自然语言文本、图像像素、视频流），需借助CNN、RNN、Transformer等深度学习技术提取特征

学习范式

有监督学习

训练集包含输入特征与已知输出标签（ X, y ），算法学习目标映射函数 $f: X \rightarrow y$ ，适用于预测任务（分类、回归）

无监督学习

训练集仅包含输入特征无标签（ X ），算法探索数据内在分布与关联，适用于描述性任务（聚类、降维、关联规则）

范式选择策略

标注数据充足时优先选用有监督学习追求精度；标注成本高或探索性分析时采用无监督学习发现潜在模式

概念汇总：数据预处理技术体系

数据预处理是连接原始数据与机器学习算法的桥梁，这四项技术共同决定模型性能上限，是数据科学家的核心技能。

数据清洗

Data
Cleaning

涵盖缺失值填补、异常值检测与处理、重复记录删除、格式统一等操作，旨在提升数据质量与一致性，减少噪声对模型的干扰。

标准化与归一化

Standardization

将不同量纲的特征转换到统一尺度（Z-score 或 Min-Max），防止大数值特征在距离计算或梯度下降中占据主导地位，加速算法收敛。

数据降维

Dimensionality
Reduction

通过 PCA 保留最大方差方向，或通过 t-SNE 保持局部结构进行可视化，有效缓解“维度灾难”，降低过拟合风险并减少计算开销。

衍生变量

Derived
Variables

基于领域知识对原始特征进行数学变换（如对数化、多项式交互、时间序列分解），生成更能揭示业务本质的新特征，往往比算法调参对模型改进效果更显著。

数据挖掘与机器学习

第二次作业

信管 2301

阮钰博

2021321054017

数据回归方法：七步标准流程

从业务问题到数学模型再到实际应用的完整转化链路，问题定义决定分析方向，数据质量决定模型上限，验证环节确保泛化能力。

- 01 问题定义与变量识别：** 明确研究目标并确定因变量 Y 与自变量 X 的因果假设关系，决定后续模型解释的业务含义与预测方向。
- 02 数据收集与质量检查：** 系统性收集样本数据并检查缺失值、异常值和变量量纲，确保输入数据满足建模假设。
- 03 探索性数据分析：** 通过散点图矩阵观察变量间关系形态，计算 Pearson 相关系数量化线性关联程度。
- 04 模型选择与假设设定：** 根据因变量类型选择一元 / 多元线性回归或 Logistic 回归，明确误差分布、线性关系等关键统计假设。
- 05 参数估计与优化：** 线性回归采用最小二乘法最小化残差平方和，Logistic 回归采用极大似然估计最大化观测数据的联合概率。
- 06 模型诊断与显著性检验：** 利用 R^2 评估解释力、F 检验判断整体显著性、t 检验验证单个系数、残差图检验异方差性与正态性假设。
- 07 模型验证与部署应用：** 通过训练集 / 测试集划分或交叉验证评估泛化性能，最终将模型用于解释变量关系或预测新样本。

线性回归模型：从一元到多元

线性回归通过寻找最佳拟合直线或超平面来量化变量间的线性关系。一元模型 $y = \beta_0 + \beta_1 x + \varepsilon$ 刻画单一因素影响；多元模型通过控制其他变量不变，分离出每个自变量的独立贡献。

一元线性回归

- 模型方程：** $y = \beta_0 + \beta_1 x + \varepsilon$ ，其中 β_0 为截距项， β_1 为回归系数量化边际效应， ε 为随机误差项。
- 参数估计：** 采用最小二乘法 (OLS)，最小化残差平方和 $\sum (y_i - \hat{y}_i)^2$ ，使拟合直线到所有样本点的垂直距离平方和最小。
- 适用场景：** 单一因素对连续型结果变量的影响分析，如温度对冰淇淋销量、学习时长对考试成绩的影响。

多元线性回归

- 模型扩展：** $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$ ， β_i 表示在其他变量恒立下， x_i 对 y 的净效应 (ceteris paribus)。
- 矩阵表达：** $Y = X\beta + \varepsilon$ ， Y 为 $n \times 1$ 向量， X 为 $n \times (p+1)$ 设计矩阵，便于计算机实现大规模数据的并行计算。
- 识别优势：** 通过统计控制混杂变量，分离目标变量的独立贡献，避免遗漏变量偏误，是观察性研究中因果推断的重要工具。

Logistic 回归：分类问题的概率模型

核心洞察： Logistic 回归通过 Sigmoid 函数将线性组合映射为概率值，解决二分类问题的预测需求。模型核心在于对数发生比 $\ln[p/(1-p)]$ 的线性表达，回归系数表示自变量对发生比的对数影响。

模型构建

$p = 1/[1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}]$ ，Sigmoid 函数将实数域线性预测值压缩至 (0,1) 区间，符合概率取值约束。

系数含义

β_i 表示 x_i 每增加一个单位，对数发生比的平均变化量； e^{β_i} 为发生比比率（OR 值），表示风险变化倍数。

参数估计

采用极大似然估计（MLE）而非最小二乘法，通过最大化观测样本的联合似然函数求解系数，因似然函数非线性，通常采用 Newton-Raphson 等迭代算法求解。

对数发生比解释

$\ln[p/(1-p)] = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ ，logit 函数建立对数 - 线性关系，使系数具有可加的边际效应解释。

分类决策

根据预设阈值（通常 0.5，可调整）将概率转化为类别标签： $p \geq \text{阈值}$ 则判为 1，否则判为 0，阈值选择可基于 ROC 曲线优化。

线性回归 vs Logistic 回归关键差异

对比维度	线性回归	Logistic 回归
因变量类型	连续数值型（如价格、温度）	二值类别型（0/1，如是 / 否）
输出范围	无界（ $-\infty, +\infty$ ）	有界（0, 1）概率值
估计方法	最小二乘法（OLS）	极大似然估计（MLE）
系数解释	自变量对 Y 的边际变化量	自变量对对数发生比的变化量

两类回归因变量类型差异决定了模型形式、估计方法与解释逻辑的根本不同。

线性回归与 Logistic 回归：联系与区别

核心洞察：两类回归同属监督学习框架，共享可解释性与多元分析能力，但因变量类型的根本差异导致了模型形式、估计方法与适用场景的分野。

核心联系

- 01 同属监督学习范式：**两者均依赖带标签的历史数据 (X,Y) 进行训练，通过学习输入到输出的映射函数实现预测，都遵循数据准备、模型训练、验证评估的标准机器学习流程。
- 02 共享可解释性优势：**回归系数具有明确的数学定义与业务含义，可量化每个自变量对因变量的边际贡献，支持因果推断与策略优化。
- 03 多元扩展能力：**均支持多自变量同时建模，可通过控制混杂变量分离目标因素的独立效应，系数估计在多元框架下保持一致的统计解释逻辑。

关键区别

- 01 因变量性质差异：**线性回归适用于连续型数值结果（如收入、温度），Logistic 回归专用于二值类别结果（如是否购买、是否违约）。
- 02 输出约束机制：**线性回归输出无界实数，Logistic 回归通过 Sigmoid 函数将输出约束在 $(0,1)$ 概率区间。
- 03 参数估计原理：**线性回归采用最小二乘法最小化残差平方和，存在解析解；Logistic 回归采用极大似然估计，需依赖数值迭代算法求解。
- 04 预测目标转化：**线性回归直接预测条件期望 $E(Y|X)$ ，Logistic 回归先预测概率 $P(Y=1|X)$ 再通过阈值判别类别。

读书笔记：《白话机器学习算法》第六章

核心洞察：回归分析的本质是通过加权线性组合寻找最佳拟合线，兼具预测精度与可解释性优势。然而，模型对异常值敏感、易受多重共线性困扰、且无法自动捕捉非线性关系。更为关键的是，回归系数仅反映统计相关关系，不能直接推导出因果结论，必须结合业务逻辑与实验设计进行因果推断。

SECTION 01

核心思想与预测机制

最佳拟合线原理：回归分析寻找使残差平方和最小的直线或超平面，单一变量形成简单趋势线，多变量通过加权组合提高预测准确度，权重即回归系数反映相对重要性。

系数解释力：标准化后的回归系数可直接比较变量重要性，正系数表示同向变化，负系数表示反向变化，绝对值大小量化影响强度，为业务决策提供量化依据。

SECTION 02

模型优势与固有局限

显著优势：计算高效、结果直观、易于向非技术人员解释，相比黑盒模型具有天然的透明性与可信度，特别适合需要解释模型决策过程的场景（如金融风控、医疗辅助诊断）。

关键局限：对异常值极度敏感，可能因个别极端样本导致回归线严重偏离；自变量高度相关时产生多重共线性，使系数估计方差膨胀；假设线性关系，无法自动拟合曲线或交互效应。

SECTION 03

方法论反思：相关与因果

相关不等于因果：回归系数仅表明变量间的统计关联强度与方向，不能证明X导致Y的因果机制，可能存在遗漏变量、反向因果或混杂因素的干扰。

模型评价的平衡： R^2 高不代表模型好，需结合残差分析、业务合理性、泛化性能综合判断；好的回归模型应既能准确预测，又能提供符合业务直觉的可解释洞察。

数据挖掘与机器学习

第三次作业

信管 2301

阮钰博

2021321054017

一、整理分类算法的基本逻辑

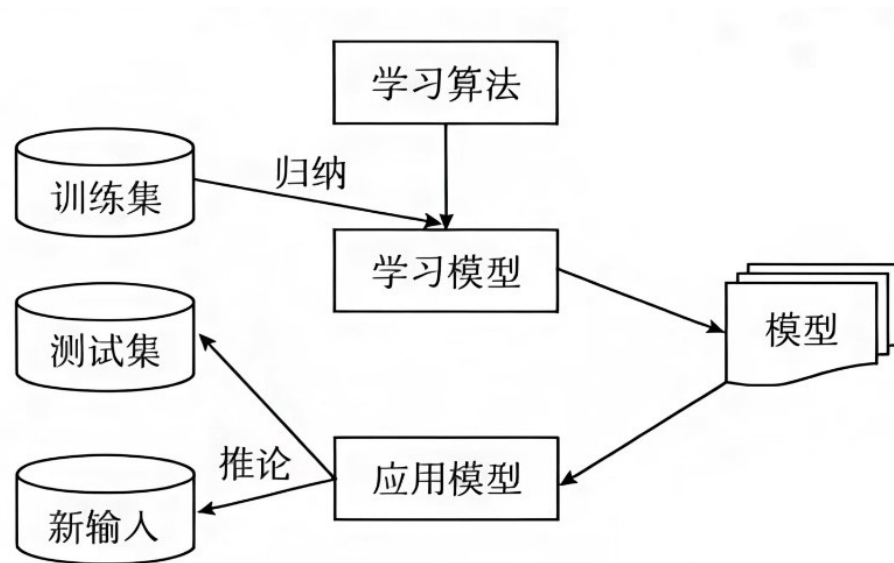


图 8-1 分类原理示意图

分类算法的基本逻辑就是利用带标签的训练数据，学习一个能够推广到新样本的判别函数或概率模型，从而在给定特征的情况下输出最可能的类别。

一、整理分类算法的几个基本概念

分类器：能够解释原始数据，能够预测新数据的模型；

训练集：用来训练模型的原始数据（划分出一部分）；

类标签：原始数据中已经给每个记录（对象）标记的类别；

测试集：用来测试模型的原始数据（划分出的一部分）；

开放集：使用训练好的模型（分类器）进行分类工作的新数据；

注意：分类是一种预测，不是肯定的，因此不能期待准确率达到100%，但是总是比较选出准确率最高的模型。

二、整理掌握 KNN 算法的思想与算法步骤

KNN 算法的思想本质是局部近似

KNN 算法基本步骤

8.2 K-近邻

8.2.1 K-近邻原理

K-近邻 (K-Nearest Neighbor, KNN) 算法是一种基于实例的分类方法, 最初由 Cover 和 Hart 于 1968 年提出, 是一种非参数的分类技术。

K-近邻分类方法通过计算每个训练样例到待分类样品的距离, 取和待分类样品距离最近的 K 个训练样例, K 个样品中哪个类别的训练样例占多数, 则待分类元组就属于哪个类别。使用最近邻确定类别的合理性用下面的谚语来说明: “如果走像鸭子, 叫像鸭子, 看起来像鸭子, 那就是鸭子。”

KNN 算法具体步骤如下:

- 步骤 1: 初始化距离为最大值。
- 步骤 2: 计算未知样本和每个训练样本的距离 $dist$ 。
- 步骤 3: 得到目前 K 个最近邻样本中的最大距离 $maxdist$ 。
- 步骤 4: 如果 $dist$ 小于 $maxdist$, 则将该训练样本作为 K-最近邻样本。
- 步骤 5: 重复步骤 2~步骤 4, 直到未知样本和所有训练样本的距离都算完。
- 步骤 6: 统计 K 个最近邻样本中每个类别出现的次数。
- 步骤 7: 选择出现频率最大的类别作为未知样本的类别。

三、读书笔记

KNN 算法笔记

参考教材：《数据挖掘：原理、方法与工具》

参考教材：《数据挖掘：原理、方法与工具》

2/21000 (图) 数据挖掘 (教学一)

0% 22

36/85

第 7 章 k 最近邻算法和异常检测

7.1 食品检测

7.2 物以类聚，人以群分 (核心思想)

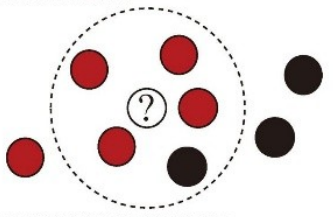


图 7-4 寻找邻居：一个数据点周围至少要有几个数据点，中心数据点才能被分类。

关键参数：k 值

k=3, k=17, k=50

(a) 过拟合 (b) 理想拟合 (c) 欠拟合

参考教材：《数据挖掘：原理、方法与工具》

参考教材：《数据挖掘：原理、方法与工具》

2/21000 (图) 数据挖掘 (教学一)

0% 22

36/85

图 7-4 寻找邻居：一个数据点周围至少要有几个数据点，中心数据点才能被分类。

关键参数：k 值

k=3, k=17, k=50

(a) 过拟合 (b) 理想拟合 (c) 欠拟合

参考教材：《数据挖掘：原理、方法与工具》

参考教材：《数据挖掘：原理、方法与工具》

2/21000 (图) 数据挖掘 (教学一)

0% 22

36/85

图 7-4 寻找邻居：一个数据点周围至少要有几个数据点，中心数据点才能被分类。

关键参数：k 值

k=3, k=17, k=50

(a) 过拟合 (b) 理想拟合 (c) 欠拟合

三、读书笔记

决策树笔记

83%

第9章 决策树

9.1 决策树是幸存者

决策树是一种机器学习模型，它通过一系列决策规则来对数据进行分类。决策树的根节点是决策的起点，通过一系列的分支（是/否）来对数据进行分类。决策树的叶子节点是决策的结果，可以是“生存”或“死亡”。

决策树的构建过程是一个递归的过程，通过不断地选择最优的分裂点来构建决策树。决策树的构建过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

决策树的构建过程是一个递归的过程，通过不断地选择最优的分裂点来构建决策树。决策树的构建过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

```
graph TD
    Root[根节点] -- 是 --> A[生存概率为0]
    Root -- 否 --> B{月收入高于5000美元吗?}
    B -- 是 --> C[生存概率为100%]
    B -- 否 --> D[生存概率为75%]
```

4/165

83%

第9章 决策树

9.3 生成决策树的过程

生成决策树的过程是一个递归的过程，通过不断地选择最优的分裂点来构建决策树。生成决策树的过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

生成决策树的过程是一个递归的过程，通过不断地选择最优的分裂点来构建决策树。生成决策树的过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

```
graph TD
    Root[根节点] -- 是 --> A{Y > 0.5吗?}
    Root -- 否 --> B[生存概率为0.5]
    A -- 是 --> C[生存概率为0]
    A -- 否 --> D[生存概率为0.9]
```

4/165

83%

第10章 随机森林

10.1 随机森林是幸存者

随机森林是一种集成学习方法，它通过构建多个决策树并将它们的结果进行平均来对数据进行分类。随机森林的构建过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

随机森林的构建过程是一个递归的过程，通过不断地选择最优的分裂点来构建决策树。随机森林的构建过程可以分为两个主要步骤：特征选择和分裂点选择。

特征选择：选择最优的特征来对数据进行分裂。常用的特征选择方法有信息增益、基尼指数和熵指数等。

分裂点选择：选择最优的分裂点来对数据进行分裂。常用的分裂点选择方法有二分法、三分法和线性扫描法等。

```
graph TD
    Root[根节点] -- 是 --> A[生存概率为0]
    Root -- 否 --> B{X > 0.25吗?}
    B -- 是 --> C[生存概率为0]
    B -- 否 --> D[生存概率为0.5]
```

4/165

数据挖掘与机器学习

第四次作业

信管 2301

阮钰博

2021321054017

簇的概念：从感性认知到量化定义

簇作为聚类分析的基本单元，其定义并非绝对精确，而是呈现出从感官层面的 "物以类聚" 到量化层面的空间邻近性的多层次理解。同一簇内对象的高相似度与不同簇间的低相似度构成了聚类的核心准则。

01 基本定义

簇是聚类过程中形成的相似对象集合，聚类本质是将物理或抽象对象划分为多个类或簇的过程，使得**类内相似性最大化**而**类间差异性最大化**。

02 理解层次

感官层面：体现为 "物以类聚，人以群分" 的直观认知；**逻辑层面**：强调簇内相似度高于簇间相似度；**量化层面**：要求同一分组记录的空间距离尽可能小，不同分组记录的距离尽可能大。

03 关键特性

簇的空间形态具有高度灵活性，可呈现球形、elongated、不规则等多种形状，其最佳划分方式高度依赖于数据的内在分布结构与分析者的期望结果。

相似性度量：欧式距离与余弦距离

核心洞察：聚类分析的核心在于量化对象间的相似性。欧式距离与余弦距离代表了两种根本不同的相似性视角：前者关注空间位置的绝对数值差异，适用于幅度敏感型数据；后者关注向量方向的一致性，适用于模式匹配型场景。



欧式距离

Euclidean Distance

- 1 衡量空间中两点的绝对几何距离，在二维平面上表现为两点间直线段的长度，计算公式为 $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- 2 关注数值大小的绝对差异，适用于物理距离、价格差异等**幅度敏感型**数据的相似性度量，对变量的量纲和量级敏感

余弦距离

Cosine Distance

- 1 衡量两个向量在方向上的差异程度，通过计算夹角余弦值 $\cos\theta = (A \cdot B) / (|A| |B|)$ 实现相似性量化
- 2 关注向量方向的一致性而非绝对大小，当 $\cos\theta$ 越接近 1 表示方向越一致，适用于文档相似度、用户偏好模式等**方向敏感型**场景

K-means 算法：基于划分的迭代优化

K-means 算法基于划分策略，通过迭代优化簇中心位置实现数据分组。算法核心在于交替执行分配步骤（将样本指派到最近中心）和更新步骤（重新计算簇均值），直至收敛条件满足，最终最小化簇内平方和（Within-Cluster Sum of Squares）。

初始化阶段：从数据集中随机选取 K 个样本点作为初始聚类中心（质心），这些初始位置对最终收敛结果有重要影响，通常需要多次随机初始化选取最优解

分配步骤：计算每个样本点到 K 个聚类中心的欧式距离，采用最近邻原则将每个样本分配到距离最近的簇，形成 K 个不相交的样本子集

更新步骤：针对每个簇重新计算所有成员点的坐标均值，将该均值向量作为新的聚类中心，使中心点始终处于簇的几何重心位置

迭代收敛：重复执行分配与更新操作，当相邻两次迭代中聚类中心的位置变化小于阈值或样本归属不再改变时算法终止，此时达到局部最优解

HC 层次聚类：构建树状层次结构

层次聚类（HC）采用自底向上的凝聚策略，通过递归合并最近邻簇构建树状层次结构，允许分析者通过在不同高度切割树结构来获得可变数量的聚类结果。

01

初始化阶段

将数据集中的每个样本点视为一个独立的簇，初始簇的数量等于样本量，每个簇仅包含单个数据点。

02

循环合并

在所有分属不同簇的样本点对中，寻找距离最近的一对，将这两个点所在的两个簇合并为一个新簇，簇的总数减一。

03

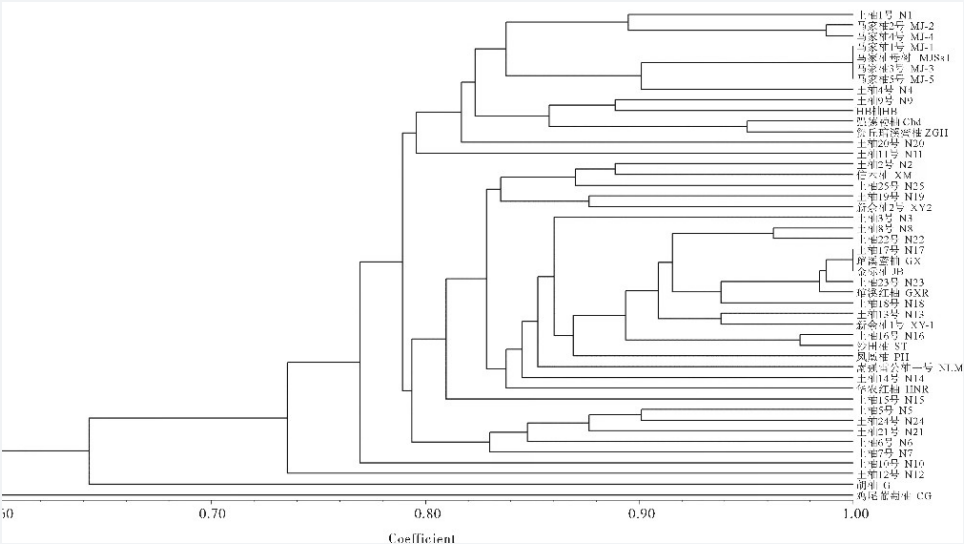
终止条件

持续执行合并操作，直到簇的总数达到预设的目标数量 K，或满足其他停止准则（如距离阈值），最终形成完整的层次结构。

04

树状图切割

通过在不同高度绘制水平切割线，可根据该线穿过的垂直连接线数量灵活确定最终聚类数目实现聚类粒度的动态调整。



层次聚类树状图（Dendrogram）与水平切割线示意

K-means 核心问题：群组数量与成员归属

"K-means 算法面临两个根本性挑战：最优群组数量的确定与群组成员的精确归属。群组数量的选择需要在模型复杂度与解释性之间权衡，过少可能掩盖数据内在结构，过多则导致过拟合；而成员分配则通过迭代优化逐步收敛到局部最优解。"

01 群组数量的主观性

K 值选择缺乏客观标准，过少可能无法揭示有意义的模式，过多则导致群组间界限模糊失去实际意义，需在模型复杂度与业务可解释性间寻求平衡。

02 陡坡图 (Elbow Method)

通过绘制 " 群组内散度 " 随 K 值增加而降低的曲线，识别曲线斜率突变处 (肘部) 作为最佳群组数量，该点之后增加群组带来的散度降低收益急剧减少。

03 成员分配的迭代机制

算法通过 " 分配 - 更新 " 的迭代循环逐步优化，使每个群组内部越来越紧凑 (成员间距离小)，群组之间越来越分离 (中心点距离大)，直至成员归属稳定。

K-means 详细步骤：从初始化到收敛

K-means 通过 " 分配 - 更新 " 的迭代循环逐步优化聚类质量。算法首先随机初始化中心点，随后进入迭代过程：计算每个样本到各中心的距离并分配至最近簇，然后基于当前簇成员重新计算中心位置。当连续迭代中无样本改变所属簇时，算法收敛至局部最优解。

01 随机初始化

从数据集中随机选择 K 个样本作为初始聚类中心（伪中心点），初始位置的选择会影响最终收敛结果，建议采用多次随机初始化或 K-means++ 改进策略

02 距离计算与分配

计算每个数据点到 K 个伪中心点的欧式距离，采用硬分配机制将每个数据点指派到距离最近的中心点所属群组，形成不相交的簇划分

03 中心点更新

针对每个簇计算所有成员点的均值向量，将该均值作为新的聚类中心，使中心始终位于簇的几何重心，最小化簇内平方和误差

04 收敛判断

当连续两次迭代中所有数据点的群组归属不再发生变化，或中心点移动距离小于预设阈值时算法终止，此时达到局部最优的聚类结果

K-means 算法局限性：适用边界分析

K-means 算法虽计算高效且易于实现，但存在三个固有局限：硬聚类机制无法表达边界样本的模糊归属；基于最小距离分配的本质导致其偏好球形簇，难以识别非凸形状；互斥划分假设使其无法处理簇间重叠或嵌套关系。

LIMITATION 01

硬聚类

Hard Clustering

每个数据点只能被分配到单一群组，采用非此即彼的刚性划分机制，无法表达位于簇边界的样本可能同时属于多个类别的模糊性。

在现实场景中，许多数据点可能处于不同类别的过渡区域，硬分配机制强制其归入单一类别，可能导致分类结果的信息损失。

LIMITATION 02

形状假设

Shape Assumption

算法基于样本到中心点的最短距离进行分配，这种最小化簇内方差的机制天然倾向于形成超球体形状的簇。

对于 elongated、月牙形、同心圆等非凸几何形状的簇结构，K-means 难以正确识别，可能将本应分开的簇合并或割裂。

LIMITATION 03

离散假设

Discreteness Assumption

算法假设簇之间相互分离、不重叠、不嵌套，无法处理存在交集或层级包含关系的复杂数据结构。

在真实数据中，不同类别往往存在重叠区域，K-means 的互斥划分假设在这种场景下会产生系统性的分类误差。

总结：聚类算法的选择策略与应用价值

聚类分析作为无监督学习的核心技术，其价值在于从海量无标签数据中自主发现隐藏的结构与模式。从相似性度量的选择到 K-means 与 HC 算法的权衡，再到对算法局限性的认知，完整的聚类实践需要结合数据特性、业务目标与计算资源进行综合决策，在模式发现与结果解释性之间寻求最优平衡。

概念基础

簇的定义具有多层次性与数据依赖性，需从业务语境出发理解 "相似" 的含义，量化层面要求簇内距离小、簇间距离大。

度量选择

欧式距离适用于幅度敏感型数据，余弦距离适用于模式匹配型场景，度量方式的选择直接影响聚类结果的业务含义。

算法权衡

K-means 适用于大规模数据的快速扁平化聚类，HC 适用于需要层次结构的可视化分析，前者需预设 K 值，后者计算复杂度较高。

局限认知

理解 K-means 的硬聚类、形状假设与离散假设局限，有助于在实际应用中选择 DBSCAN、GMM 等更适合复杂数据结构的替代算法。

数据挖掘与机器学习

第五次作业

信管 2301

阮钰博

2021321054017

1、关联选股的背景知识

行业关联选股法是把关联规则算法应用到股票市场分析中，用来发现行业或股票之间的联动关系。

股票市场中，不同行业之间往往存在产业链关系、政策影响关系、资金流关系和市场情绪关系。例如，新能源汽车行业上涨时，锂电池、充电桩、稀有金属等行业可能也会联动上涨；房地产行业上涨时，建材、家电、银行等行业也可能受到影响。

关联选股的基本思想是：把每个交易日看作一个事务，把行业或股票看作项，把当天上涨或满足条件的行业记为 1，不满足条件的记为 0。然后使用 Apriori 算法找出经常共同出现的行业组合，并进一步分析行业之间的关联规则。

这种方法可以辅助发现行业联动关系，为行业轮动、板块分析和选股提供参考。但它只能说明统计关联，不能直接证明因果关系。

2、数据分析：矩阵行与列的含义

第五讲文件夹中的对应数据文件是一个典型的 0-1 矩阵示例。表中行是事务，列是项，元素是 0 或 1。

在行业关联选股中也可以使用同样的矩阵结构：

行表示交易日。每一行代表一天的市场情况，也就是一个事务。

列表示行业或股票。每一列代表一个行业或一只股票。

矩阵元素表示该行业或股票当天是否满足条件。

例如：

1 表示该行业当天上涨，或涨幅超过设定标准。

0 表示该行业当天没有上涨，或没有达到设定标准。

如果某一天新能源汽车、锂电池、充电桩三个行业都上涨，那么这一行中这三个行业对应的值为 1，其他未上涨行业对应的值为 0。

这种矩阵的意义是把现实中的交易数据转化为 Apriori 算法可以处理的事务数据。

3、运行程序

运行文件夹中的 Python 程序。虽然它使用的是商品数据，但程序逻辑可以直接迁移到行业关联选股中。

程序主要步骤如下：

第一，导入相关库，包括 pandas、TransactionEncoder、apriori、networkx 和 matplotlib。

第二，读取 products.csv 数据。

第三，把每一行中的“商品 1、商品 2、商品 3”转化为一个事务列表。

第四，使用 TransactionEncoder 把事务数据转化为 0-1 矩阵。

第五，调用 Apriori 算法计算频繁项集。程序中设置的最小支持度是 0.02，表示某个组合至少出现 2 次。第六，只保留 2 项集，并统计共现次数。

第七，使用 networkx 构建商品关联网络图。

第八，把结果保存到 product_associations.csv。

如果应用到行业关联选股中，只需要把程序中的“商品”替换为“行业”：

products.csv 可替换为行业涨跌数据。

“商品 1、商品 2、商品 3”可替换为某交易日上涨的行业列表。

每一行商品组合可理解为某一天共同上涨的行业组合。

Apriori 输出的频繁项集可理解为经常共同上涨的行业组合。

知识图谱中的节点表示行业，边表示两个行业之间存在共现

关系，边的权重表示共同出现次数。

4、程序结果分析与思考

- 程序运行后会生成关联结果，记录了商品之间的共现关系。共现次数越高，说明两个商品在
- 同一事务中同时出现的次数越多，关联程度相对更明显。
- 例如结果中“无线鼠标和硬盘”共现 5 次，“电动牙刷和笔记本电脑”共现 5 次，“笔记本
- 电脑和路由器”共现 4 次。这说明在当前样本数据中，这些商品组合出现得比较频繁，可以认为它们之间存在一定关联。
- 如果把这个过程应用到行业关联选股中，就可以把商品替换成行业，把一次购物记录替换成一个交易日。若某两个行业经常在同一天上涨，就说明它们可能存在联动关系。例如新能源汽车和锂电池经常共同上涨，可能是因为它们处在同一产业链中，容易受到相同政策、资金或市场情绪影响。
- 但需要注意，关联规则只能说明“经常一起出现”，不能直接说明因果关系。共现次数高不一定代表投资价值高，还要结合支持度、置信度、提升度以及行业背景进行判断。尤其是提升度，如果大于 1，说明这种关联比随机情况更强；如果接近 1，说明关联可能并不明显。因此，行业关联选股法可以作为辅助分析工具，用来发现行业之间的联动关系，但不能单独作为买卖股票的依据。

数据挖掘与机器学习

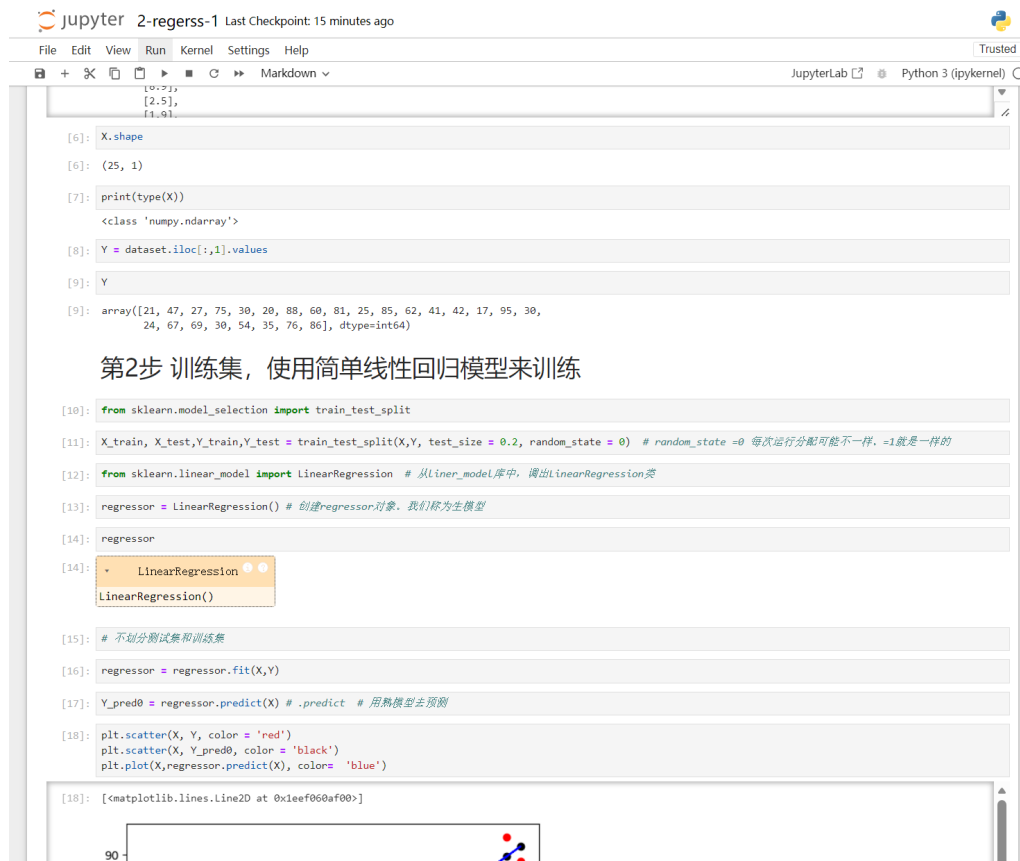
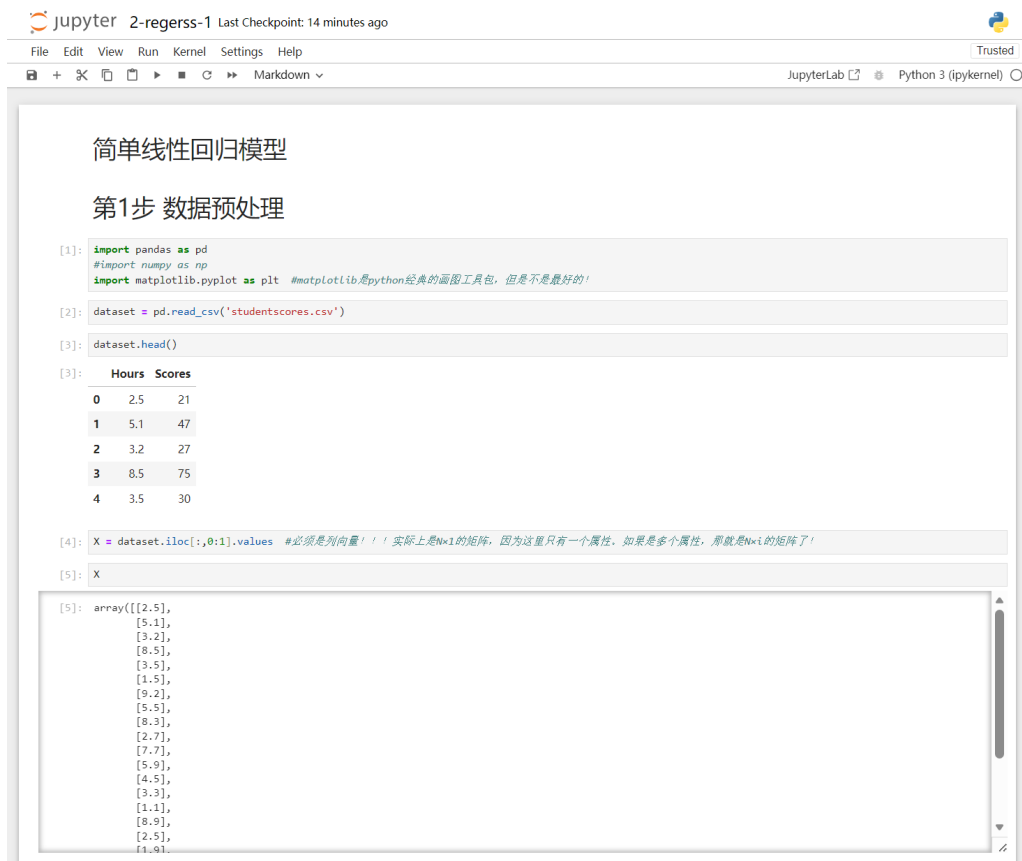
第六次作业

信管 2301

阮钰博

2021321054017

代码运行结果



代码运行结果

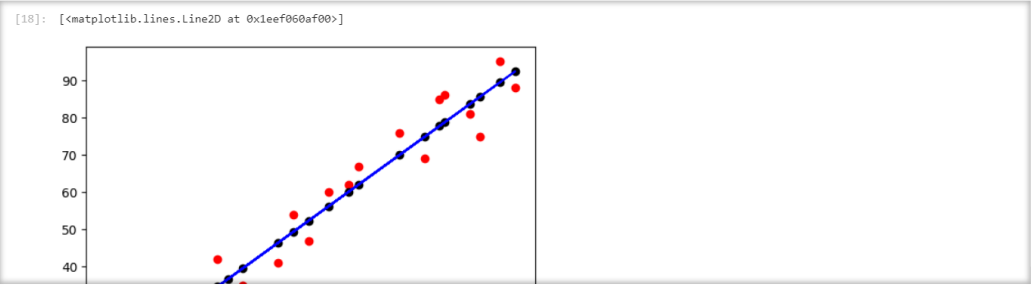
jupyter2-regerss-1Last Checkpoint: 15 minutes ago

FileEditViewRunKernelSettingsHelp

Trusted

Python 3 (ipykernel)

```
[18]: plt.scatter(X, Y, color = 'red')
      plt.scatter(X, Y_pred0, color = 'black')
      plt.plot(X,regressor.predict(X), color= 'blue')
```



```
[19]: r2 = regressor.score(X,Y)
      r2
```

```
[19]: 0.9529481969048356
```

```
[20]: # 使用划分训练集、测试集的方法来处理线性回归问题
```

```
[21]: regressor = regressor.fit(X_train,Y_train) # .fit # 训练之后，就成了熟模型 # 问“X_train 多少行？”
```

```
[22]: regressor
```

LinearRegression
LinearRegression()

第3步 预测结果

```
[23]: Y_pred = regressor.predict(X_test) # .predict # 用熟模型去预测
```

```
[24]: Y_pred
```

```
[24]: array([16.88414476, 33.73226078, 75.357018 , 26.79480124, 60.49103328])
```

第4步 可视化

jupyter2-regerss-1Last Checkpoint: 16 minutes ago

FileEditViewRunKernelSettingsHelp

Trusted

Python 3 (ipykernel)

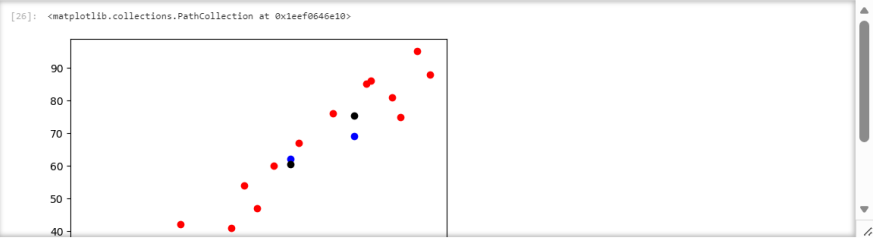
```
[24]: Y_pred
```

```
[24]: array([16.88414476, 33.73226078, 75.357018 , 26.79480124, 60.49103328])
```

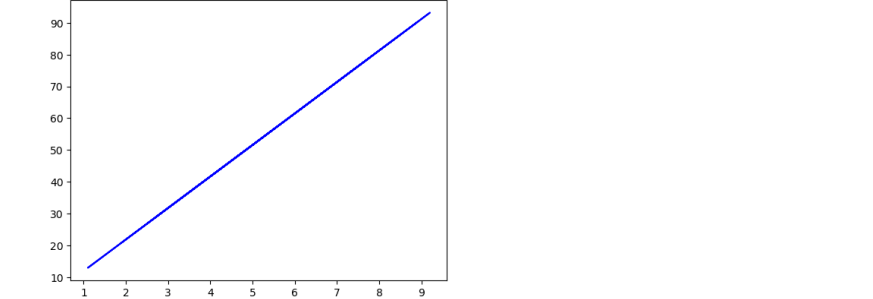
第4步 可视化

```
[25]: # 训练集结果可视化
      # 特别注意 scatter和plot的区别。
```

```
[26]: plt.scatter(X_train, Y_train, color = 'red')
      plt.scatter(X_test, Y_test,color = 'blue')
      plt.scatter(X_test, Y_pred,color = 'black')
```



```
[27]: plt.plot(X_train, regressor.predict(X_train),color = 'blue')
```

```
[27]: [

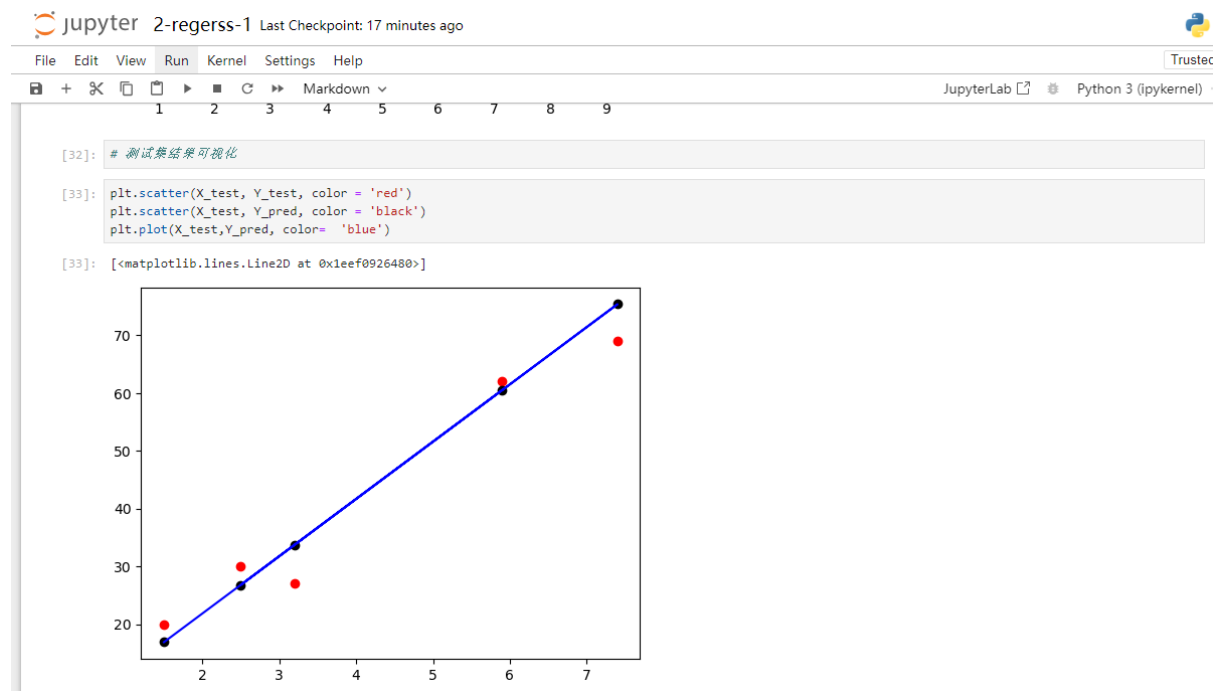
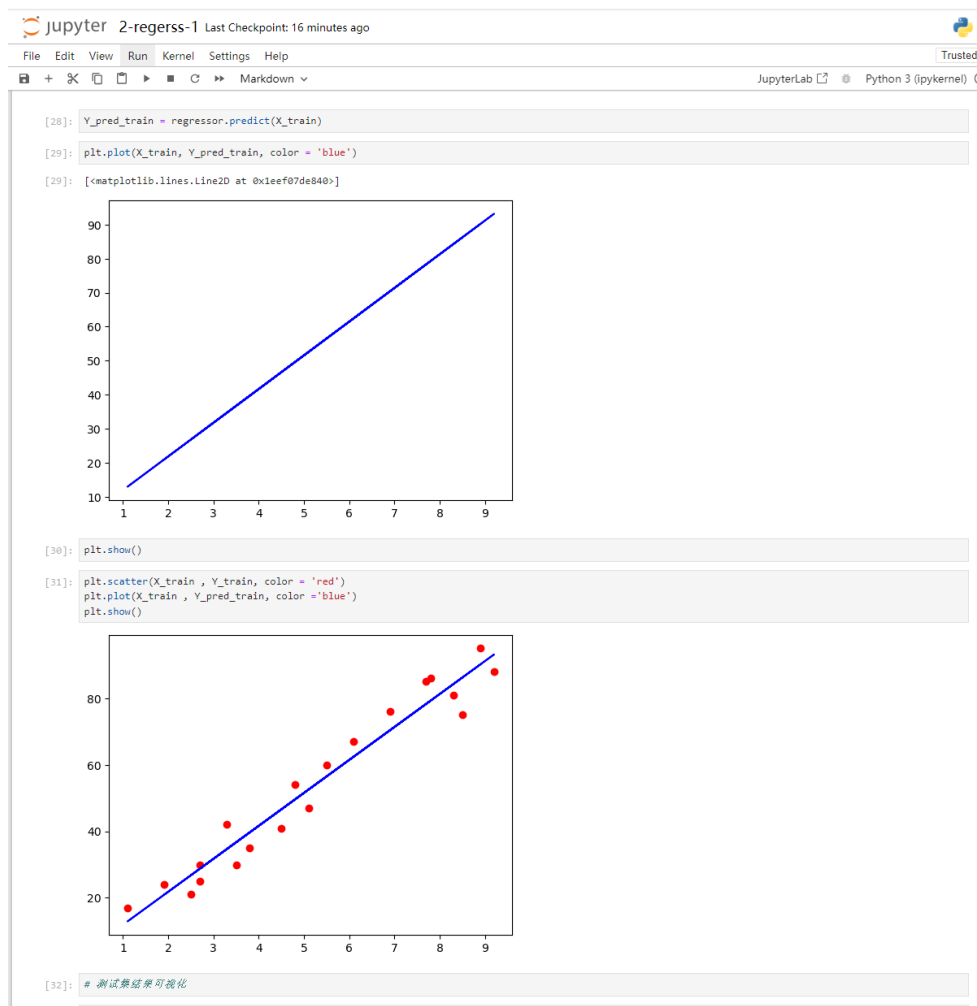
```
[28]: Y_pred_train = regressor.predict(X_train)
```



```
[29]: plt.plot(X_train, Y_pred_train, color = 'blue')
```


```


代码运行结果

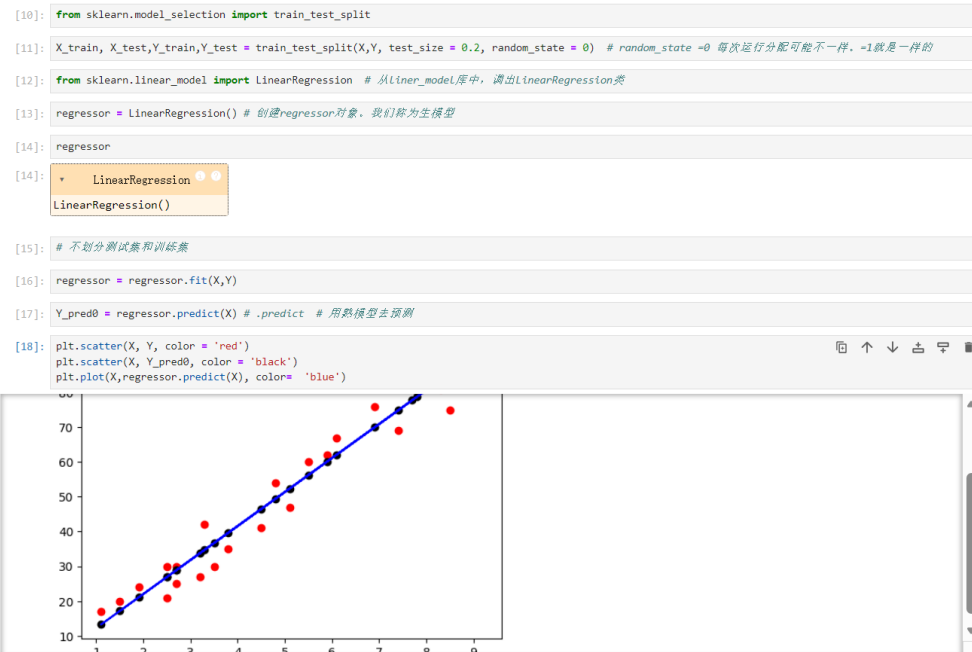


一、整理（数据处理的小细节）为什么 X 必须是列向量？变成行向量会怎么样呢？

X 实际上是一个矩阵，这个矩阵的大小是 25×1 ，25 是行数，1 是特征数，这样可以运算！如果有 M 行， N 个变量（属性 / 特征的话），那么这个矩阵就是： $M \times N$

二、如何显示生成的“熟模型”？ $y = b_0 + b_1 * x$, b_0 和 b_1 怎么获得？

第2步 训练集，使用简单线性回归模型来训练



- b_0 和 b_1 怎么生成

```
[34]: b1 = regressor.coef_ # 系数
b1
```

```
[34]: array([9.91065648])
```

```
[35]: b0 = regressor.intercept_ #截距 y = 2 + 9.9x
b0
```

```
[35]: 2.018160041434662
```

三、如何评价结果的好坏？（代码） R^2 的结果是什么？

在简单线性回归模型中，评价结果好坏的指标包括 R^2 （决定系数）、均方误差（MSE）、均方根误差（RMSE）、平均绝对误差（MAE）等。其中， R^2 是最常用的指标之一，它衡量了模型对数据变异的解释程度。

R^2 接近 1：说明自变量 x 能够很好地解释因变量 y 的变异，模型拟合效果好。

R^2 接近 0：说明自变量 x 几乎无法解释因变量 y 的变异，模型拟合效果差。

- 3 评价结果的好坏 R^2

```
[36]: R2_train = regressor.score(X_train,Y_train)
      R2_train
```

```
[36]: 0.9515510725211552
```

```
[37]: R2_test = regressor.score(X_test,Y_test)
      R2_test
```

```
[37]: 0.9454906892105354
```

四、调整训练 / 测试的比例，对结果 (R^2) 有什么影响？试试看

1. 训练集比例过小的影响

当训练集比例过小时，比如只占 20%，模型能学习到的数据规律有限。这就好比你看只看了少量的例题就去参加考试，很难掌握全部知识点。此时，模型在训练集上的 R^2 可能看起来还不错，但在测试集上的 R^2 会比较低，因为模型没有充分学习到数据中的真实关系，出现过拟合的可能性较小，但容易出现欠拟合。

2. 训练集比例过大的影响

要是训练集比例过大，例如达到 90%，模型在训练集上会学习得非常充分，甚至可能会把数据中的噪声也当作规律来学习。这就像你把练习题的答案都背下来了，但遇到新的考试题（测试集）就不会做了。这种情况下，训练集上的 R^2 会很高，接近 1，但测试集上的 R^2 可能会明显下降，出现过拟合现象。

3. 合适的训练 / 测试比例

一般来说，常见的训练 / 测试比例是 70%/30%、80%/20% 或者 75%/25%。这些比例能让模型充分学习数据规律和避免过拟合之间取得较好的平衡。不过，具体选择哪种比例，还需要根据数据集的大小和特点来决定。如果数据集很小，可能需要采用交叉验证的方法来更准确地评估模型性能。

数据挖掘与机器学习

第七次作业

信管 2301

阮钰博

2021321054017

数据集结构与探索性分析

数据字典与变量类型

连续型变量： R&D Spend、Administration、Marketing Spend 三个运营投入指标，以及因变量 Profit。

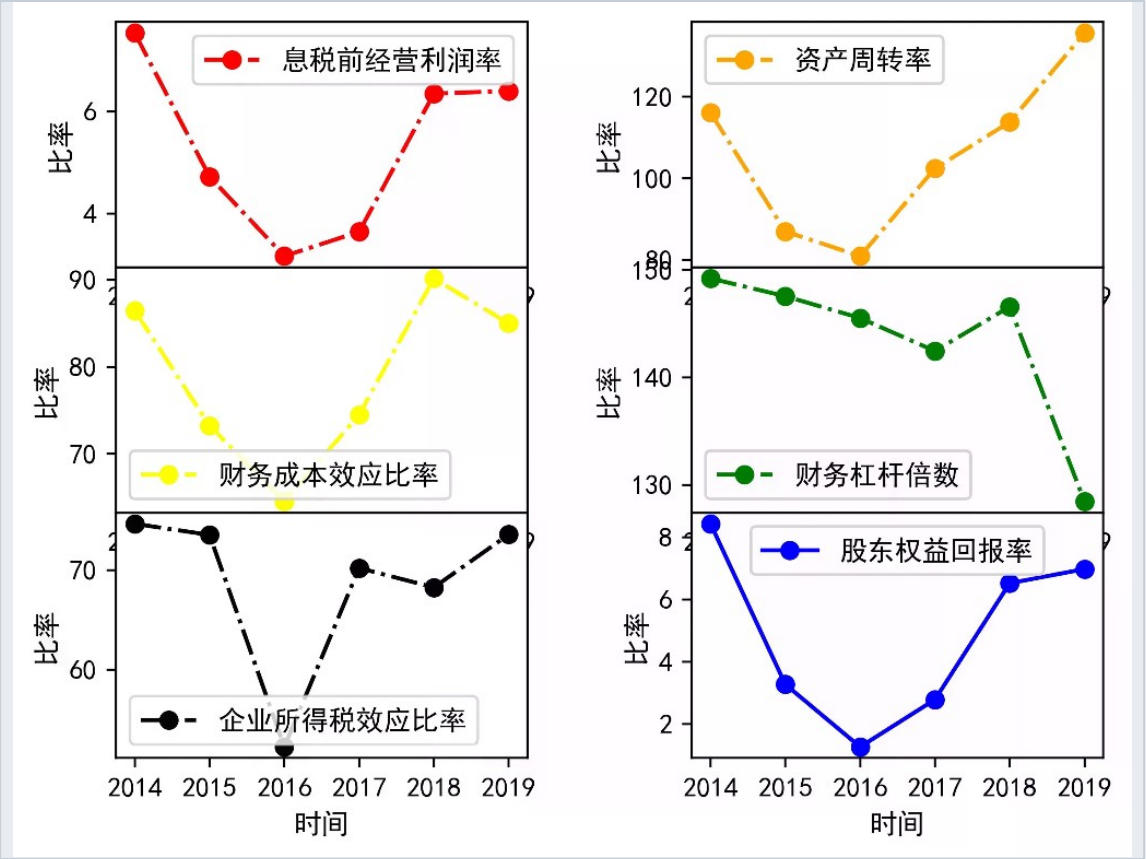
分类变量： State 包含 New York、California、Florida 三类，需经 One-Hot 编码后方可入模，避免标签编码引入虚假顺序关系。

城市因素对利润的影响

地域差异显著： 箱线图显示 New York 与 California 企业的利润中位数显著高于 Florida，组内变异较大，地理位置确实影响企业盈利能力。

多元因素并存： 同一城市内部企业利润差异显著，暗示除地域外，研发投入等运营因素同样关键，多元回归模型能分离各因素的独立贡献。

" 纽约和加州企业的利润中位数明显高于佛罗里达，这为 One-Hot 编码的必要性提供了直观证据，也预示着地理位置因素将成为模型的重要解释变量。 "



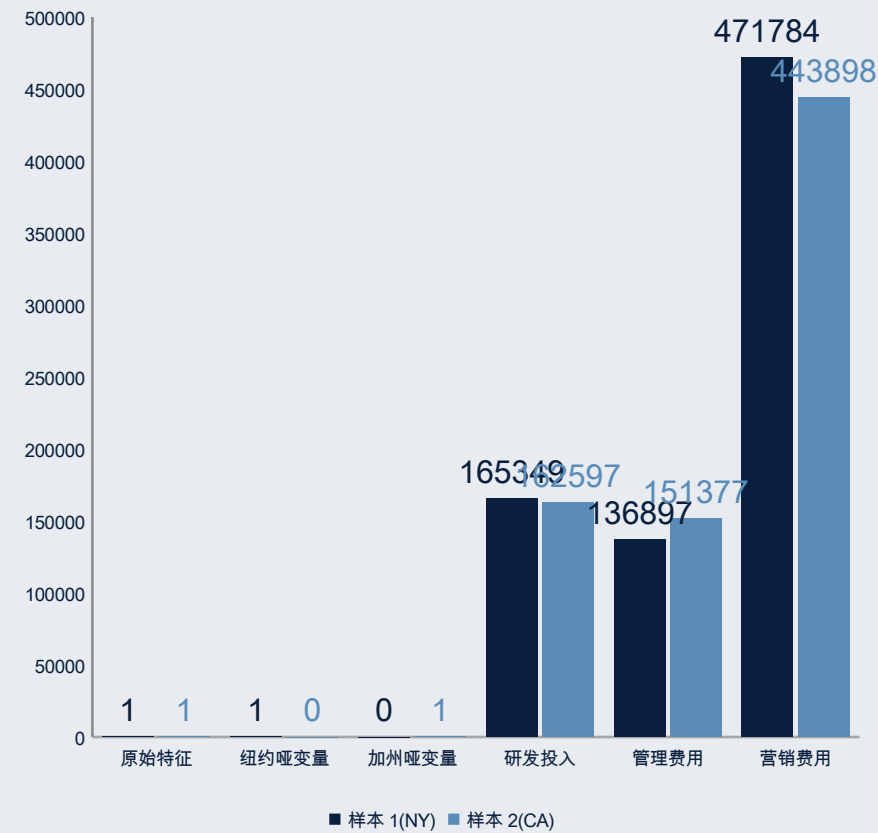
不同城市企业利润分布箱线图对比

One-Hot 编码技术路径

无序分类变量 State 必须经过 One-Hot 编码转化为数值形式，但直接 Label 编码会引入虚假顺序关系导致系数失真。正确流程包括：标签编码过渡、独热编码转换、手动删除基准列规避虚拟变量陷阱，最终得到可用于多元回归的无共线性数值特征。

- ✓ **识别分类特征：** State 为无序名义变量，类别间无高低顺序，直接标签编码（ 0/1/2 ）会使模型误判数值大小关系，导致回归系数严重偏差
- ✓ **One-Hot 编码实施：** 通过 ColumnTransformer 为每个城市创建独立的 0-1 特征列，样本属于该城市则标记为 1，否则为 0，完整保留类别信息
- ✓ **虚拟变量陷阱规避：** N 个分类生成 N 个独热列会导致完全多重共线性，必须手动删除 1 个基准类别列（ 代码中 $X = X[:,3:]$ ），确保模型可识别
- ✓ **最终数据集构建：** 经过编码处理后，所有特征均为纯数值型，无虚假顺序、无共线性问题，可直接用于多元线性回归训练

One-Hot 编码后的特征矩阵结构示意图



One-Hot 编码将分类变量转化为多个二元特征，配合基准类别删除策略避免共线性

模型构建与评估：拟合优度分析

多元线性回归模型展现出优异的拟合性能， $R^2=0.949$ 表明模型可解释 94.9% 的利润变异。截距项 42989 代表基准利润水平，各投入变量的回归系数揭示了研发投入对利润的贡献强度远超管理与营销费用，印证了技术创新驱动企业盈利的理论预期。

模型性能评估

$R^2 = 0.949$ 解释方差比例

模型解释力接近 95%，表明四个自变量（含两个城市哑变量）共同捕获了利润变化的主要模式，残差随机性较好。

预测能力验证：拟合优度接近 0.95 说明模型具有良好的预测能力，可用于解释变量关系并进行新样本的利润预测。

回归方程与系数估计

最终数学公式

$$\hat{Y} = 42989.01 + 0.7788 \times R\&D + 0.0294 \times Admin + 0.0347 \times Marketing + \beta_4 \times N$$
$$Y + \beta_5 \times CA$$

截距项解读：42989 代表所有投入为 0 时企业的基准利润水平。

边际效应比较：研发投入每增加 1 单位，利润平均增加 **0.78 单位**，显著高于其他成本项（管理 0.03、营销 0.03），凸显技术创新的价值创造能力。

回归系数深度解读

回归系数揭示了创业企业利润形成的差异化驱动机制：研发投入呈现极强的正向效应（系数 0.7788），是利润增长的核心引擎；管理与营销投入仅具微弱促进作用（系数约 0.03）；城市因素则体现为基准利润水平的区位差异。

核心引擎

研发投入

系数高达 **0.7788**，表明研发投入每增加 1 单位，利润平均增长 0.78 单位，转化效率极高。

研发强度决定企业技术创新能力，是创业公司在激烈市场竞争中建立护城河的关键，应作为资源配置的优先方向。

边缘贡献

管理营销

管理费用系数 **0.0294**、营销费用系数 **0.0347**，仅为研发系数的 1/20，对利润拉动效果十分有限。

过度行政管理造成效率损耗，粗放式营销投入 ROI 偏低，提示企业应优化管理流程、精准营销而非简单追加预算。

基础差异

城市因素

纽约和加州相对于佛罗里达具有显著的基础利润优势，反映区域经济环境、产业集群效应或市场成熟度差异。

地理位置虽不可控，但企业可通过入驻高利润区域或调整区域战略来利用这种基础差异。

方法反思：分类变量处理的科学性



关键洞察：分类变量的预处理方式是保障多元线性回归科学性的关键前提。直接 Label 编码会人为制造虚假的大小顺序关系，导致回归系数严重偏差；One-Hot 编码配合虚拟变量消除策略，才能在保留类别信息的同时避免共线性，确保模型结果准确可靠。这一技术细节体现了数据科学中 "Garbage In, Garbage Out" 的核心原则。

1 标签编码的风险

将纽约、加州、佛罗里达映射为 0、1、2，模型会默认 $2 > 1 > 0$ 的数值关系，错误判定类别间存在等级顺序，导致回归系数、权重、预测结果全部严重失真。

2 One-Hot 编码的必要性

为每个类别创建独立的二元特征列，确保类别间相互独立、无大小关系，模型能够正确估计各类别相对于基准组的差异效应。

3 虚拟变量陷阱的规避

N 个类别生成 N 个独热列会导致设计矩阵列向量线性相关（完全多重共线性），必须删除一个基准类别作为参照，使模型矩阵满秩、系数可识别。

4 有序与无序的区分

仅当分类变量本身存在天然顺序（如低 / 中 / 高等级）时才考虑 Label 编码；对于城市这类名义变量，必须采用 One-Hot 编码以保证建模科学性。

数据挖掘与机器学习

第八次作业

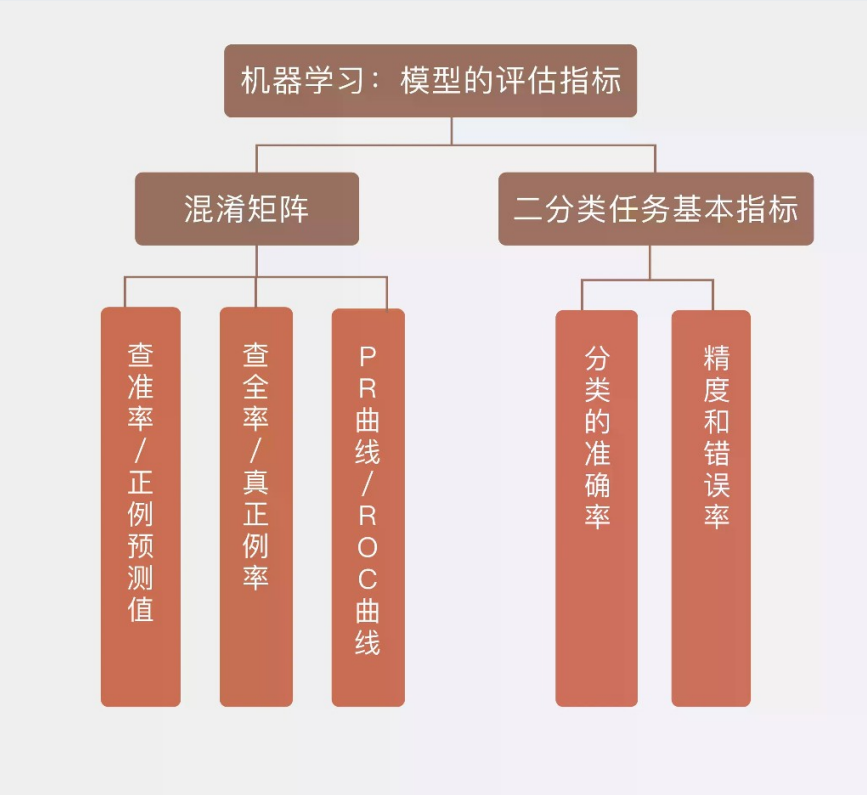
信管 2301

阮钰博

2021321054017

特征工程：性别因素的 One-Hot 编码处理

CONFUSION MATRIX



混淆矩阵结构示意图
TP/FP/TN/FN 四象限分布

■ 未加入性别因素

- 01 基线模型仅使用原始数值特征进行训练，通过混淆矩阵计算得准确率 **0.95**，真阳性 55 例、真阴性 21 例，误判 7 例（假阳性 3 例、假阴性 1 例）
- 02 混淆矩阵呈现 **[[55,3],[1,21]]** 的对角优势分布，说明基线模型已具备较强的分类能力

+ 加入性别 One-Hot 编码

- 01 将性别变量（男 / 女）转化为 One-Hot 编码的两个二元特征列，避免标签编码（0/1）引入的**虚假大小关系**导致的系数偏差
- 02 处理后模型准确率仍为 **0.95**，混淆矩阵结构与基线一致，说明性别因素在该购买预测任务中区分度有限，可能与其他特征存在信息重叠

💡 **关键洞察：**性别变量的 One-Hot 编码处理确保了模型能够正确理解分类变量的名义属性，避免标签编码引入虚假顺序关系。实验结果显示，在该数据集上加入性别因素后模型准确率维持在 0.95 不变，表明性别并非该分类任务的关键预测因子，但 One-Hot 编码的技术规范性仍不可或缺。

模型对比：KNN 与 Logistic 回归性能评估

" 在相同数据集与划分策略下，KNN 算法 (准确率0.95) 显著优于 Logistic 回归 (准确率0.925)。KNN 在识别正例 (购买) 方面表现突出 (真阳性 21 vs 17)，而 Logistic 回归存在更多的假阴性误判 (5 vs 1)，暗示该数据集的决策边界可能更适合非线性模型拟合。 "

算法	真阳性 (TP)	假阴性 (FN)	假阳性 (FP)	真阴性 (TN)	准确率
KNN (K=3)	21	1	3	55	0.95
Logistic 回归	17	5	1	57	0.925

数据来源：混淆矩阵对比实验结果 (n=80)

✓ KNN 漏报风险低

KNN (K=3) 混淆矩阵呈现 [[55,3],[1,21]] 的分布，假阴性仅 1 例，说明对潜在购买客户的识别能力较强。

Logistic 漏报严重

Logistic 回归混淆矩阵为 [[57,1],[5,17]]，假阴性高达 5 例，存在较严重的漏报问题，可能错失营销机会。

负例识别相当

两类模型均保持较高的真阴性识别率 (55 vs 57)，在排除非购买客户方面性能相当。

📊 准确率计算

对角线元素之和除以总样本量，KNN 达到 $(55+21)/80=0.95$ ，Logistic 为 $(57+17)/80=0.925$ ，差距 2.5 个百分点。

超参数调优：训练集比例与 K 值敏感性分析

超参数调优揭示了模型复杂度过高导致的过拟合风险与数据划分策略的关键影响。K=1 时模型过度拟合训练噪声（准确率跌至 0.8875），而 $K \geq 3$ 时通过邻域平均机制有效抑制噪声，准确率稳定在 0.95；训练集比例在 $\text{test_size}=0.2$ 时达到最优平衡，过小导致训练不充分，过大则测试集代表性不足。

训练集比例优化

test_size=0.1：准确率未达最优，训练样本过多导致测试集代表性不足，评估结果波动较大

test_size=0.2：准确率达峰值 0.95，实现训练充分性与测试可靠性的最佳平衡，为后续实验的标准划分策略

test_size $\geq$ 0.3：准确率下降至 0.916（0.3）和 0.906（0.4），训练数据不足导致模型欠拟合

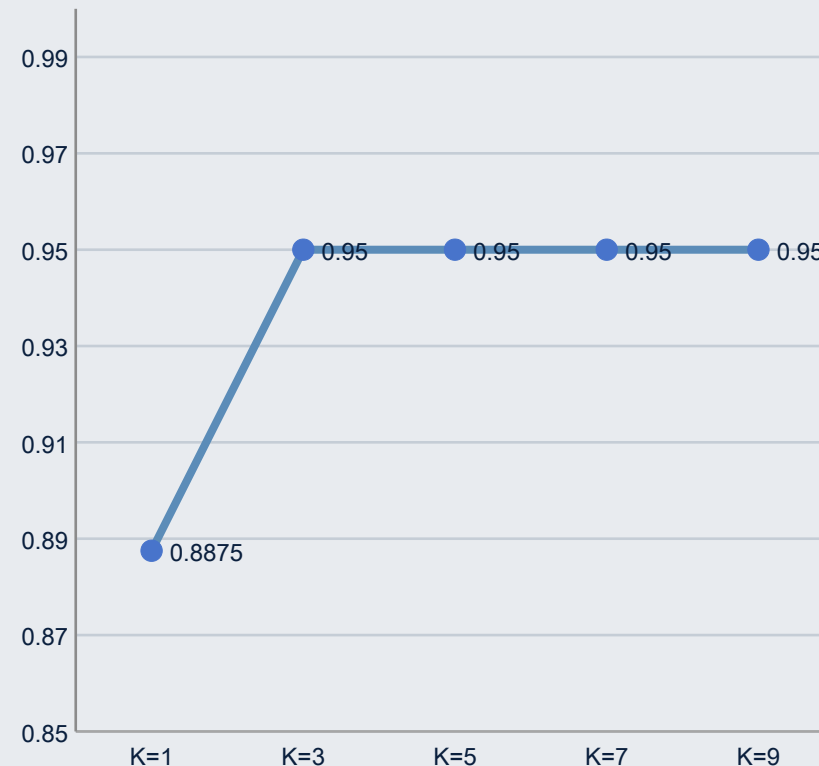
K 值选择与过拟合控制

K=1：准确率仅 0.8875，决策边界不规则锯齿状，对噪声点极度敏感，典型过拟合

K=3：准确率跃升至 0.95，邻域平均机制有效平滑噪声，模型复杂度降低，泛化性能显著改善

K $\geq$ 5：准确率稳定在 0.95，适度 K 值（3-9）均能取得最优性能，K 值选择具有一定容错空间

K 值与模型准确率关系



K=1 时准确率显著下降， $K \geq 3$ 后稳定在 0.95 水平

实验结论与实践启示

💡 本实验系统验证了分类模型开发中的关键技术决策：特征工程的技术规范性优先于性能提升，算法选择应匹配数据分布特性（非线性边界优选 KNN），超参数调优需平衡拟合能力与泛化性能（ $K \geq 3$ 且 $test_size=0.2$ 为较优配置），混淆矩阵比单一准确率更能揭示模型的类别识别偏差。

- 01 特征工程的技术规范性：** One-Hot 编码虽在本案例中未提升准确率（0.95 vs 0.95），但消除了标签编码的虚假顺序风险，是分类变量预处理的标准做法，体现 "Garbage In, Garbage Out" 的数据质量原则
- 02 算法选择的场景适配：** KNN（0.95）在该数据集上显著优于 Logistic 回归（0.925），特别是在正例识别上（TP 21 vs 17），提示对于可能存在非线性决策边界的分类任务，应优先尝试基于实例的学习算法
- 03 过拟合防范与复杂度控制：** $K=1$ 时准确率跌至 0.8875 且决策边界不规则，验证了 "最近邻过少导致过拟合" 的理论预期； $K \geq 3$ 时通过邻域投票机制有效抑制噪声，准确率稳定在 0.95，证明适度的模型正则化不可或缺
- 04 数据划分的策略优化：** $test_size=0.2$ 时模型表现最稳定，过小（0.1）导致评估不可靠，过大（0.4）造成训练不足，该发现与机器学习实践中 "80/20 划分" 的经验法则高度吻合
- 05 评估指标的多维解读：** 混淆矩阵揭示 Logistic 回归存在 5 例假阴性，可能错失营销机会，而 KNN 仅 1 例；单一准确率指标可能掩盖类别不平衡下的识别偏差，业务场景中应结合精确率、召回率综合评估

数据挖掘与机器学习

第九次作业

信管 2301

阮钰博

2021321054017

实验内容概览

01 特征工程实验

通过 LabelEncoder 将性别变量编码为 0/1，构建包含 Age、EstimatedSalary 和 Gender 的三维特征矩阵，对比原始二维特征与增强特征的分类准确率差异。

02 特征缩放影响

验证决策树算法对 StandardScaler 标准化处理的敏感性，比较标准化前后（Age 与 EstimatedSalary 原始值 vs 缩放值）的模型准确率与混淆矩阵差异。

03 多模型对比

在相同训练集划分（test_size=0.2）和标准化处理条件下，对比 KNN(k=5)、Logistic 回归、决策树和随机森林四种算法的准确率与混淆矩阵表现。

04 决策树可视化

限制树深度为 3 层，展示根节点（Age 分裂）到叶节点的完整决策路径，解读基尼指数 / 信息熵变化与样本分布规律。

核心洞察： 本实验围绕决策树算法展开四个维度的探索：特征工程验证性别因素对分类性能的贡献度，方法论层面揭示决策树对特征缩放的不敏感性，横向对比四种经典算法在同一数据集上的表现差异，并通过可视化技术解析决策树的内部决策机制，形成从实践到理论的完整认知闭环。

特征工程实验：性别因素对决策树性能的影响

实验结果表明，加入性别特征后决策树准确率从 0.9250 下降至 0.9125，混淆矩阵显示假阳性增加 1 例、假阴性增加 2 例。这一现象揭示了特征工程的核心原则：并非所有特征都能提升模型性能，与目标变量弱相关或无关的特征可能引入噪声，增加模型复杂度并导致过拟合。

- ✓ **基线模型**：原始特征组合 (Age+EstimatedSalary) 在测试集上达到 0.9250 的准确率，混淆矩阵呈现 [[55,3],[1,21]] 的分布，仅 4 例误判，基线模型表现优异
- ✓ **特征增强**：采用 LabelEncoder 对性别进行数值编码 (Female=0, Male=1)，构建三维特征矩阵 X_gender，经 StandardScaler 标准化后训练决策树分类器
- ↓ **性能下降**：增强特征后的模型准确率降至 0.9125，混淆矩阵变为 [[54,4],[3,19]]，总误判数从 4 例上升至 7 例，其中假阴性误差增长更为显著 (从 1 例增至 3 例)
- 💡 **原因分析**：准确率下降的原因在于性别变量与购买决策的关联性较弱，可能与其他特征存在信息重叠，决策树在分裂过程中优先选择 Age 和 EstimatedSalary，性别成为冗余特征反而增加了过拟合风险

特征组合准确率对比



加入性别特征后准确率下降 1.25 个百分点

特征缩放对决策树性能的影响分析

核心发现：实验验证了决策树算法对特征缩放的不敏感性——在标准化处理与原始数据两种条件下，模型准确率均为 0.9000，混淆矩阵完全一致。这一特性源于决策树基于阈值分裂的机制仅依赖特征的相对排序而非绝对数值尺度。

01 实验设计

- 使用 StandardScaler 对 Age 和 EstimatedSalary 进行标准化处理（均值为 0，标准差为 1），对比原始数据与标准化数据在相同 random_state 下的模型表现
- 保持训练集划分比例（test_size=0.2）和决策树参数（criterion='entropy', random_state 固定）完全一致，确保实验的可比性

02 实验结果

- 不缩放条件下的准确率为 0.9000，混淆矩阵为 [[53,5],[3,19]]；缩放后的决策树准确率同样为 0.9000，混淆矩阵分布完全一致
- 两次训练生成的树结构完全相同，验证了在固定随机种子前提下，决策树对线性缩放变换具有不变性

03 机理解释

- 决策树的分裂规则仅关心特征值的相对顺序（如 Age<=30 vs Age<=0.5），只要数据划分相同，不同尺度下的分裂阈值会相应调整，找到的样本分割完全等价
- 这一特性使得决策树在特征工程阶段无需进行标准化处理，简化了数据预处理流程，但需注意极端异常值仍可能影响分裂点选择

多模型性能对比：四种分类算法评估

KNN 与 Logistic 回归

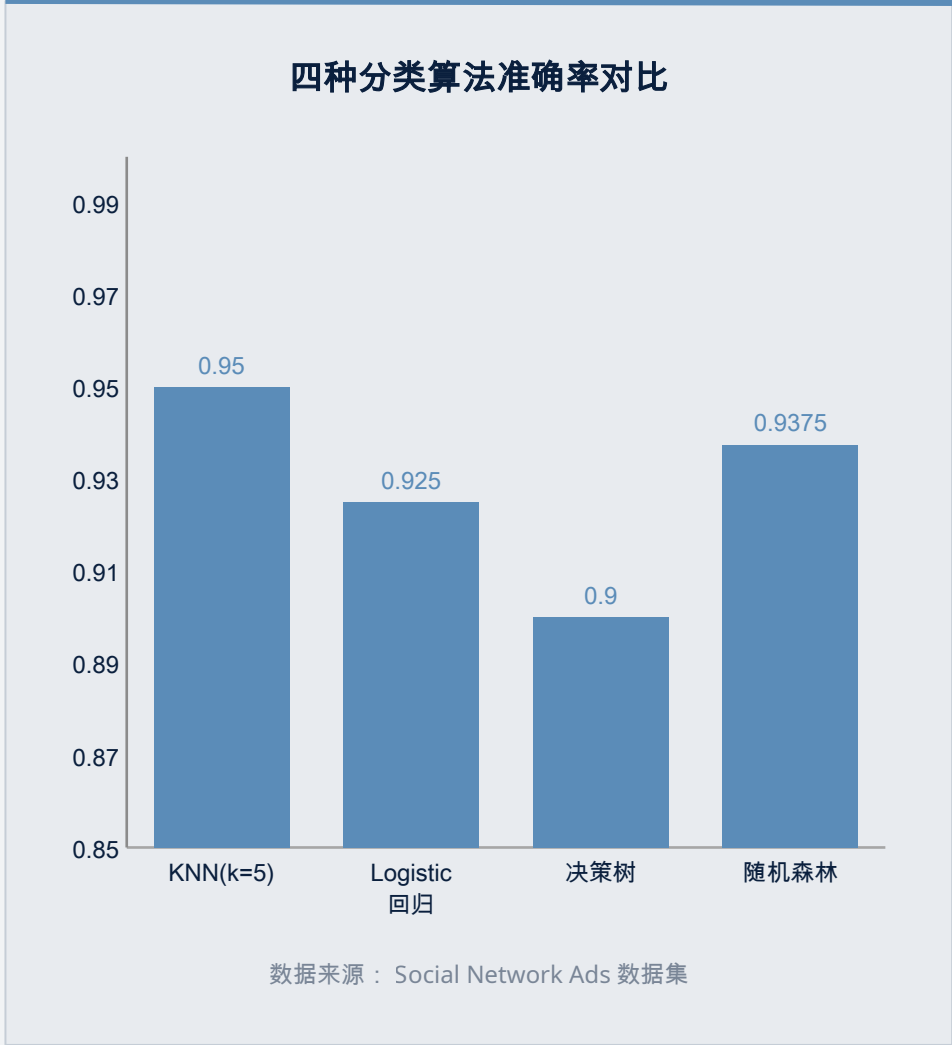
KNN(k=5) 准确率达 0.9500，混淆矩阵 $\begin{bmatrix} 55 & 3 \\ 1 & 21 \end{bmatrix}$ 显示假阴性仅 1 例，对潜在购买客户识别能力强，适合非线性决策边界场景。

Logistic 回归 准确率 0.9250，但混淆矩阵 $\begin{bmatrix} 57 & 1 \\ 5 & 17 \end{bmatrix}$ 显示假阴性高达 5 例，漏报风险较大，其线性假设限制了在该数据集上的表现。

决策树与随机森林

单棵决策树 准确率 0.9000，混淆矩阵 $\begin{bmatrix} 53 & 5 \\ 3 & 19 \end{bmatrix}$ ，存在过拟合风险，对训练数据敏感，泛化能力相对较弱。

随机森林 ($n_estimators=100$) 准确率 0.9375，混淆矩阵 $\begin{bmatrix} 55 & 3 \\ 2 & 20 \end{bmatrix}$ ，通过 Bagging 集成有效降低方差，在准确率与稳健性间取得最佳平衡。



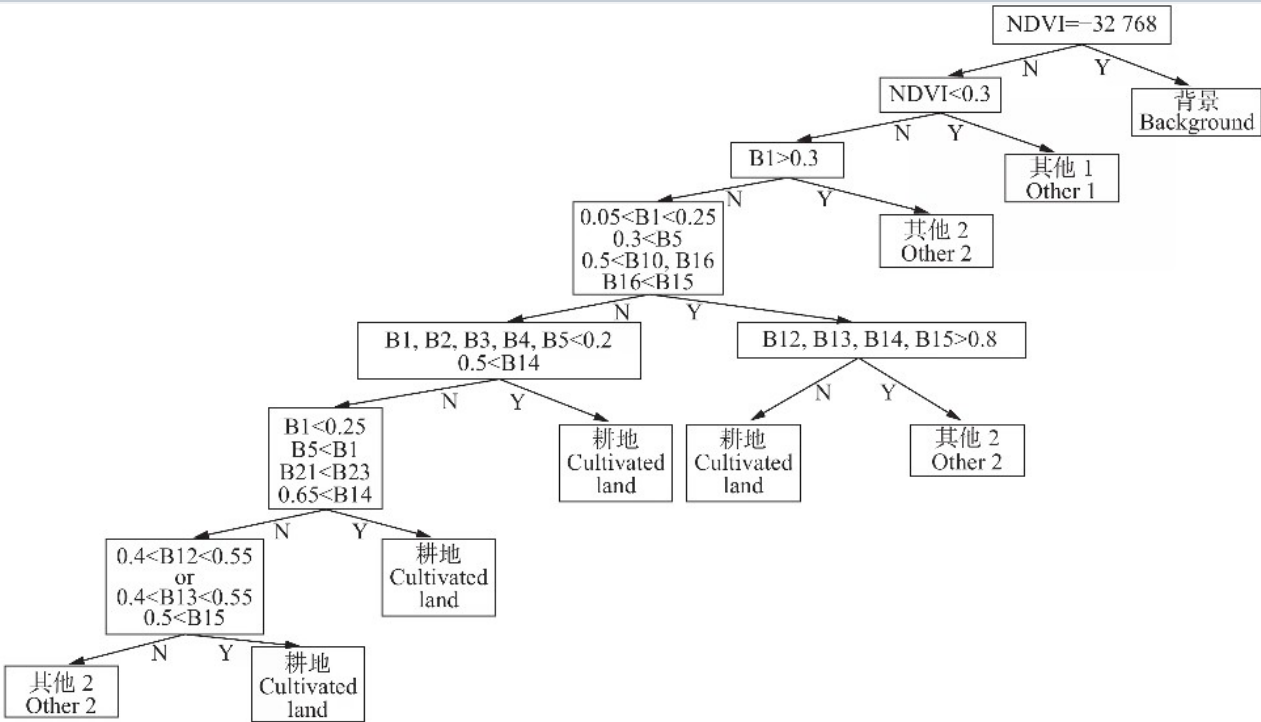
决策树结构可视化与决策路径分析

根节点分裂：基于 $\text{Age} \leq 0.611$ （标准化后约 35 岁），样本从总样本 320 例分流为左子树 228 例与右子树 92 例，信息熵从 0.957 开始下降。

左分支纯化：年龄较大群体中， $\text{EstimatedSalary} \leq 0.596$ 的节点包含 186 例样本，其中 178 例未购买，信息熵降至 0.256，纯度极高。

右分支细分：年轻高收入群体 ($\text{EstimatedSalary} > -0.824$) 按 $\text{Age} \leq -0.167$ 二次分裂，识别出 28 例年轻高收入购买者。

模型洞察：决策树通过年龄与收入的阈值组合进行硬分类，叶节点中部分纯度不高 ($\text{entropy}=0.722$)，提示单棵树存在过拟合风险。



决策树 (max_depth=3) 结构可视化：展示根节点分裂、信息熵变化及样本分布路径

" 树结构揭示了年龄与收入的交互作用对购买决策的关键影响，但单层决策树的分裂规则较为简单，难以捕捉特征间的复杂非线性关系。 "

实验结论与方法论反思

" 本实验系统验证了决策树算法的特性与机器学习实践中的关键权衡：特征工程需遵循相关性原则而非简单堆砌；决策树的尺度不变性简化了预处理但需警惕过拟合；算法选择应权衡偏差-方差与业务场景；可视化技术有助于理解模型决策逻辑但不应替代定量评估。 "

特征工程策略

特征选择应基于与目标变量的相关性评估，性别变量的冗余性导致准确率下降 1.25%，验证了 "少即是多" 的特征工程原则，避免维度灾难与过拟合。

算法特性认知

决策树对特征缩放的不敏感性 (0.9000 vs 0.9000) 体现了基于排序分裂的机制优势，但单棵树易受噪声干扰，随机森林通过集成将准确率提升至 0.9375，揭示了集成学习的价值。

模型选择建议

在 Social Network Ads 数据集上，KNN(k=5) 以 0.9500 的准确率表现最优，但需权衡计算复杂度；随机森林在准确率与稳健性间取得最佳平衡，适合作为生产环境的基线模型。

数据挖掘与机器学习

第十次作业

信管 2301

阮钰博

2021321054017

一、用花萼的数据聚类

```
[39]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

# 加载数据
iris = load_iris()
X = iris.data # 所有4列数据
X_sepal = iris.data[:, 0:2] # 花萼数据: 第0列(花萼长度)和第1列(花萼宽度)
X_petal = iris.data[:, 2:4] # 花瓣数据: 第2列和第3列

print("数据形状:", X.shape)
print("花萼数据形状:", X_sepal.shape)

数据形状: (150, 4)
花萼数据形状: (150, 2)

[40]: # 使用花萼数据 (sepal length, sepal width)
estimator_sepal = KMeans(n_clusters=3, random_state=42)
estimator_sepal.fit(X_sepal)

# 获取聚类标签和中心
labels_sepal = estimator_sepal.labels_
centers_sepal = estimator_sepal.cluster_centers_

print("\n=== 花萼数据聚类结果 ===")
print("聚类中心:\n", centers_sepal)
print("前10个样本的聚类标签:", labels_sepal[:10])

# 可视化
plt.figure(figsize=(16, 6))

plt.subplot(1, 3, 1)
# 分别绘制三个簇
for i in range(3):
    mask = labels_sepal == i
    plt.scatter(X_sepal[mask, 0], X_sepal[mask, 1],
                label=f'Cluster {i}', alpha=0.7, s=50)

# 绘制聚类中心
plt.scatter(centers_sepal[:, 0], centers_sepal[:, 1],
            c='black', marker='x', s=200, linewidth=3, label='Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Clustering on Sepal Data (K=3)')
plt.legend()
plt.grid(True, alpha=0.3)
```



仅凭花萼特征难以完美区分所有样本，若加入花瓣特征可显著提升聚类效果。

二、用所有数据聚类

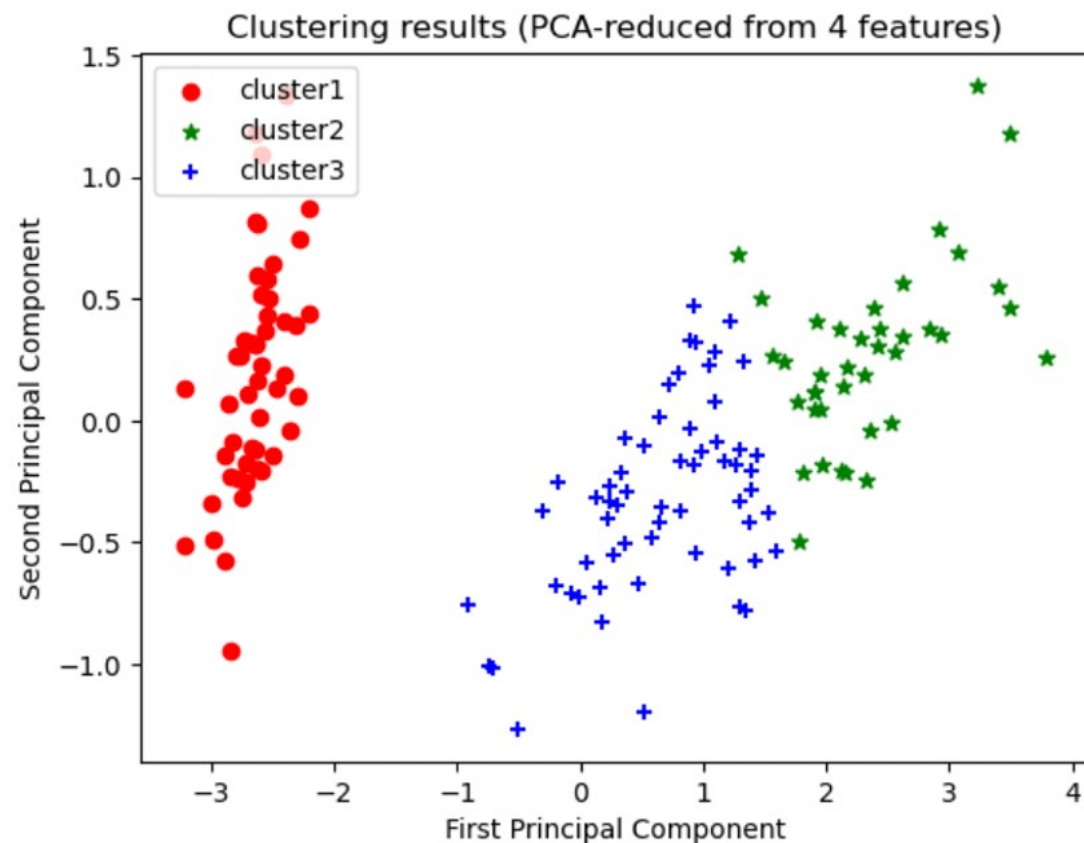
```
#使用PCA降维到2D进行可视化
from sklearn.decomposition import PCA

X_3 = iris.data[:, :] # 使用全部4个特征
estimator.fit(X_3)
label_pred = estimator.labels_

# PCA降维到2维
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_3)

x0 = X_pca[label_pred == 0]
x1 = X_pca[label_pred == 1]
x2 = X_pca[label_pred == 2]

plt.scatter(x0[:, 0], x0[:, 1], c='red', marker='o', label='cluster1')
plt.scatter(x1[:, 0], x1[:, 1], c='green', marker='*', label='cluster2')
plt.scatter(x2[:, 0], x2[:, 1], c='blue', marker='+', label='cluster3')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('Clustering results (PCA-reduced from 4 features)')
plt.legend(loc=2)
plt.show()
```



红色簇（Setosa）完全独立，分离度极高；另外两簇虽有部分重叠，但整体界限清晰。这说明 $K=3$ 的选择合理，算法成功捕捉到了数据在高维空间中的自然分组特征。

三、这里的 K=3 ，把 K 改成 2 ，试一试

```
[42]: # 使用花瓣数据，但将K改为2
estimator_sepal_k2 = KMeans(n_clusters=2, random_state=42)
estimator_sepal_k2.fit(X_sepal)

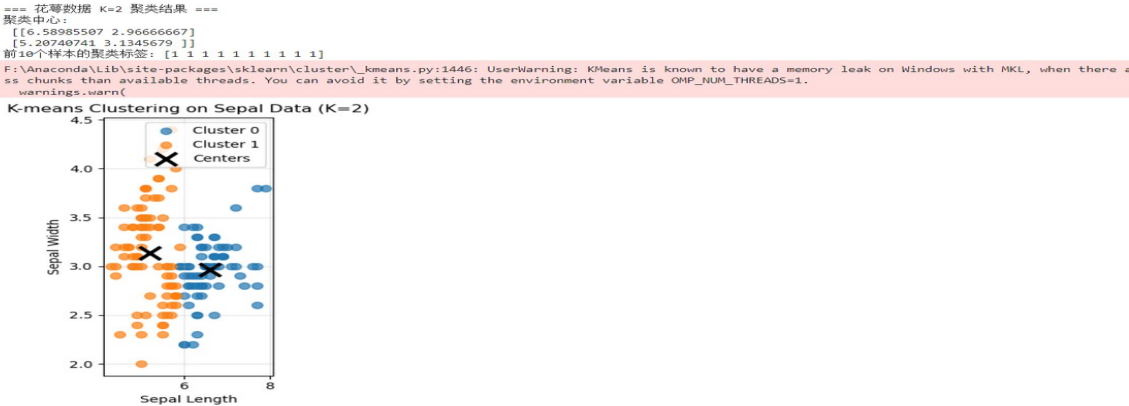
labels_sepal_k2 = estimator_sepal_k2.labels_
centers_sepal_k2 = estimator_sepal_k2.cluster_centers_

print("\n=== 花瓣数据 K=2 聚类结果 ===")
print("聚类中心:\n", centers_sepal_k2)
print("前10个样本的聚类标签:", labels_sepal_k2[:10])

# 可视化 K=2 的结果
plt.subplot(1, 3, 3)
for i in range(2):
    mask = labels_sepal_k2 == i
    plt.scatter(X_sepal[mask, 0], X_sepal[mask, 1],
               label=f'Cluster {i}', alpha=0.7, s=50)

plt.scatter(centers_sepal_k2[:, 0], centers_sepal_k2[:, 1],
           c='black', marker='x', s=200, linewidth=3, label='Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Clustering on Sepal Data (K=2)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



四、查考资料，分析该案例，聚类的其他输出，比如：聚类中心（代码）。可视化聚类中心。

```
[43]: # 详细分析聚类中心
print("\n" + "="*50)
print("聚类中心详细分析")
print("="*50)

# 对比不同K值和不同特征集的聚类中心
print("\n1. 花萼数据 K=3 的聚类中心:")
for i, center in enumerate(centers_sepal):
    print(f"    Cluster {i}: 花萼长度={center[0]:.2f}, 花萼宽度={center[1]:.2f}")

print("\n2. 所有数据 K=3 的聚类中心 (前两个维度):")
for i, center in enumerate(centers_all):
    print(f"    Cluster {i}: 花萼长度={center[0]:.2f}, 花萼宽度={center[1]:.2f}")

# 可视化聚类中心 (更清晰的展示)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
# 绘制原始数据点 (用颜色区分真实类别, 以便对比)
plt.scatter(X_sepal[:, 0], X_sepal[:, 1], c=iris.target,
            cmap='viridis', alpha=0.5, s=50)
plt.scatter(centers_sepal[:, 0], centers_sepal[:, 1],
            c='red', marker='*', s=300, edgecolors='black', linewidth=2, label='K-means Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Centers on Sepal Data\n(Colored by True Labels)')
plt.legend()
plt.colorbar(label='True Species')

plt.subplot(1, 2, 2)
# 绘制 K=2 的聚类中心
plt.scatter(X_sepal[:, 0], X_sepal[:, 1], c=labels_sepal_k2,
            cmap='viridis', alpha=0.7, s=50)
plt.scatter(centers_sepal_k2[:, 0], centers_sepal_k2[:, 1],
            c='red', marker='*', s=300, edgecolors='black', linewidth=2, label='K=2 Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Centers (K=2)\n(Colored by Cluster Labels)')
plt.legend()
plt.colorbar(label='Cluster Label')

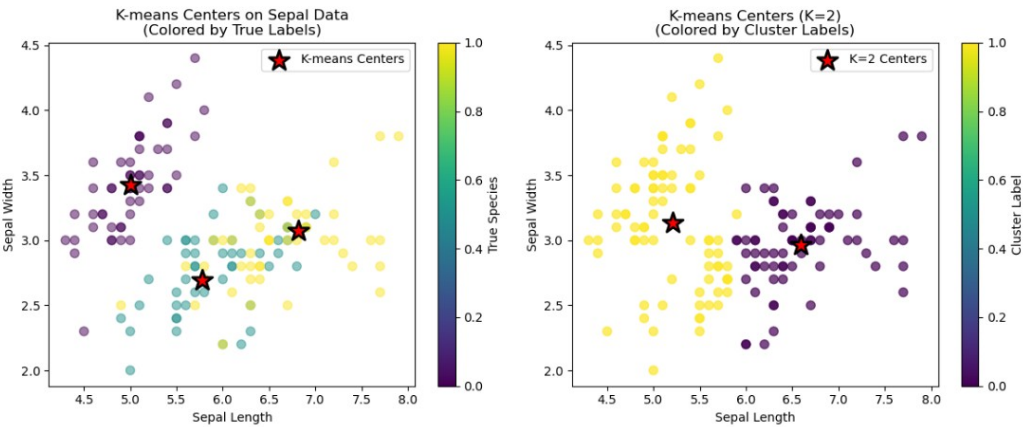
plt.tight_layout()
plt.show()

# 计算并比较不同模型的 Inertia (簇内误差平方和)
print("\n不同模型的 Inertia (簇内误差平方和, 越小越好):")
print(f"花萼数据 K=3: {estimator_sepal.inertia_.2f}")
print(f"所有数据 K=3: {estimator_all.inertia_.1.2f}")
print(f"花萼数据 K=2: {estimator_sepal_k2.inertia_.2f}")

# 分析聚类效果
print("\n聚类效果分析:")
print("- 当K=3时, 花萼数据将样本分为3组, 但可以看到某些簇重叠较多")
print("- 当K=2时, 算法将样本分为两大类, 左下角一组, 右上角一组")
print("- 使用所有4维数据时, 聚类效果通常比仅用2维更好 (inertia更小)")
print("- 聚类中心表示每个簇的平均特征, 可以用来代表该簇的典型样本")
```

=====
聚类中心详细分析
=====

- 1. 花萼数据 K=3 的聚类中心:
Cluster 0: 花萼长度=6.81, 花萼宽度=3.07
Cluster 1: 花萼长度=5.77, 花萼宽度=2.69
Cluster 2: 花萼长度=5.01, 花萼宽度=3.43
- 2. 所有数据 K=3 的聚类中心 (前两个维度):
Cluster 0: 花萼长度=6.85, 花萼宽度=3.08
Cluster 1: 花萼长度=5.01, 花萼宽度=3.43
Cluster 2: 花萼长度=5.88, 花萼宽度=2.74



不同模型的 Inertia (簇内误差平方和, 越小越好):
花萼数据 K=3: 37.05
所有数据 K=3: 78.86
花萼数据 K=2: 58.21

- 聚类效果分析:
- 当K=3时, 花萼数据将样本分为3组, 但可以看到某些簇重叠较多
 - 当K=2时, 算法将样本分为两大类, 左下角一组, 右上角一组
 - 使用所有4维数据时, 聚类效果通常比仅用2维更好 (inertia更小)
 - 聚类中心表示每个簇的平均特征, 可以用来代表该簇的典型样本